GERUNDELLE: A Formally Verified Architecture of Mind

Twelve Layers, from World Physics to Narrative, Machine-Checked in Lean 4

Scott Seto

Independent Researcher

October 2026

Abstract

Where should a mind's correctness live? In a neural network trained end to end, it lives in behavior: the network is trained to keep the distinctions a mind must keep (between what it saw and what it imagined, between what it lived through and what it was told), but nothing guarantees that it does. Compartmentalization in neural networks is an open problem. GERUNDELLE places them instead in the mathematical structure of the mind's operators, where they can be stated exactly and proved.

Gerundelle is a twelve-layer architecture of mind whose promises are theorems. Its layers run from a world that conserves its one physical quantity, through perception, competition for attention, provenance of evidence, memory, understanding and self-awareness, to ownership, affect, planning, social exchange and the selection of what is worth telling. Each layer carries guarantees that a proof assistant (Lean 4) has checked for every possible input, every possible sequence of operations, every possible interleaving. The development comprises **24 files**, **301 theorems**, and **no unfinished proofs**.

The architecture is domain-independent: one mind and one algebra run in story, code, chess and population worlds, with no domain-specific machinery, and only the world changes. The domains are projections, not separate systems. Because most of the guarantees are stated over the mind and the algebra rather than over any domain, they hold wherever the mind is placed.

Three results carry the paper.

1. Every layer is verified. From conservation of potential in the world to the well-foundedness of a told story, each guarantee is derived from an invariant the system's own design documents or tests already state. What the system promises, the proofs check.

2. Six fields, one structure. Six areas of mathematics that rarely cite one another (information security, the geometry of symmetry groups, parallel transport in fiber bundles, the theory of how concepts can be learned in sequence, a way of tracking what operations have been performed, and a calculus of evidence and trust), developed independently over more than a century, from Klein's Erlangen program (1872) to graded monads (2014), turn out to describe the same architecture once stated about its own operators and data. Each becomes a guarantee about a different layer. Bridge theorems connect them: each states a result proved in one setting about another layer's own operators, ledger or data routing.

3. Seeing and feeling are one property. What an agent perceives and what it feels are both instances of the same mathematical structure: factorization through a view. Perception factors through the Umwelt (the world simplified by the agent's invariances). Affect factors through the ownership register (the territory the agent perceives as its own). A theorem proves the connection, and proves which emotions break it and why.

We do not claim the underlying mathematics as new. What is new is that these results (from six fields, across twelve layers) hold of one system, are connected by proved bridges, and are machine-checked end to end. Formalizing the system also found a defect in its reference implementation, now corrected: the test suite had passed with the bug in place.

To our knowledge, GERUNDELLE is the first architecture of mind in which every layer, from perception through affect, planning, social exchange and narrative selection, carries machine-checked guarantees, and the first in which those guarantees are connected across layers by proved bridge theorems.

Our aim is to demonstrate that a formally verified, unified architecture of mind is possible, and to exhibit one.

1 Introduction

1.1 The Problem: Chains That Break

The standard way to represent knowledge in a machine is a chain of rules: *if A then B; if B and C then D*. The chain is clear, checkable, and brittle. The moment your understanding of B changes (you discover B has a hidden condition, or B was never quite what you thought), the chain breaks. Every conclusion that depended on B must be retracted and rebuilt. Add a new condition and the chain breaks again. This is not a bug in any particular system. It is the nature of chains: they transmit force in one direction, and a link that changes shape shatters every link below it.

Worse: what the chain does not know, it cannot represent. An unknown is a gap (a variable with no value, a query that returns nothing). The gap has no shape. It exerts no pull. It cannot tell the system what kind of evidence would fill it, or which of its neighbors are closest to answering it.

Every architecture of mind makes promises about how it handles these problems. Imagination is not evidence. What was told cannot become what was lived by thinking about it. Attention is scarce. A story depends only on what came before. These promises are checked, if at all, by running the system on the sequences someone thought to try.

No test suite covers every sequence. We argue that an architecture of mind can be held to the standard of mathematics: its promises stated as theorems, and checked by a proof assistant for every sequence at once.

1.2 The Gerundelle System

GERUNDELLE is a twelve-layer architecture of mind built on three ideas.

The world is a tree of doings. A jar does not have mass, volume and a chemical composition. It is *charged*: it holds potential drawn from a spring. A boulder is *resting*: its weight balances the ground's support. The world has no coordinates or distances. Each object is described by what it is doing (a *gerund*), and a doing is a balance of external forces against internal reserves such as grip, capacity or seal. When a force crosses a reserve the gerund flips: *charged* becomes *empty*, *resting* becomes *falling*. Where a configuration invites a force that has not been applied (a button to be pressed, a lever with no one at its long arm), the world has an *open force slot*: the question "what would happen if the force were applied?" is itself a structure, the same kind of open variable the mind keeps about everything it does not yet understand. The ontology is Whitehead's: things are what they are doing [Whitehead, 1929]. The vocabulary of forces and reserves follows Talmy's force dynamics, an excavation of how people already represent causation [Talmy, 1988]. And an open force slot computes what Gibson described as an affordance [Gibson, 2014].

Cognition is a market. A mind is a marketplace whose scarce resource is commitment: at any point, a slot can be filled by one thing, not many. Hypotheses, patterns and wants compete

for slots, and the competition is settled by evidence from acting in the world, not by enumeration, derivation or gradient. Each candidate's bid is computed from its trials and their provenance: what the agent lived through counts in full, what it was told counts at a discount, and what it imagined counts not at all.

A market cognition algorithm is as fundamental a computational primitive as an evolutionary algorithm. An evolutionary algorithm produces solutions that work. A market cognition algorithm produces solutions that know *why* they work: every concluded holding carries the receipts that funded it, and every receipt names the world interaction that created it.

A slot held is territory, funded by receipts, and a contradiction from the world reopens it. Understanding is carried by the same structure: an SOV (*Structured Open Variables*) is a graph of concepts connected by relations (caused by, conditions, role), some concepts open and some filled, each open concept closing when evidence funds it past a threshold and reopening when the world contradicts it. Not when it is derived. When the world has pushed back often enough. Ignorance is not absence but the shape of the open slots: what the mind does not know, structured by what it does.

Ownership is the foundational motivation. What the agent acts for is territory: whatever it perceives as its own, a boundary its pushes fill and whose effects it can predict. An object persists while its internal forces balance the external ones; an agent is an object that acts to keep and extend that balance. Every want traces back to an ownership slot, and understanding is purchased only as far as some ownership slot needs it.

Emotions are how this territory reads. Frustration is a claim that keeps failing. Curiosity is an unclaimed region winning attention. Joy is a claim that took. These are not labels applied after the fact. They are states of the ownership register, read off its graph, and a theorem proves which inputs each one depends on.

1.3 What This Paper Does

The goal of the program this paper belongs to is to relocate much of the burden of cognitive correctness from learned runtime behavior to the mathematical structure of a mind's operators. Rather than training a system to approximate the desired relationships, it defines those relationships explicitly, proves properties of their composition, and allows complex behavior to arise through their interaction with evidence and the world. Humans are the existing example of this division of labor: much of what a mind must get right is given by its structure, and a life supplies the evidence.

This differs from the two established approaches in what it fixes in advance. Symbolic AI fixes the answers: what is known about the world is written as chains of rules, which shatter when a condition changes (section 1). Statistical AI fixes very little, and learns how to handle evidence in the same weights as what the evidence says about the world, so it holds its own invariants (that imagining is not seeing, that hearsay is not experience) only statistically, and cannot guarantee them: keeping one kind of content apart from another is an open problem for neural networks. Models fail to keep the instructions they are given apart from the data they read, and prompt engineering and fine-tuning do not reliably close the gap [Zverev et al., 2025]; deployed systems have been compromised by exactly this failure [Greshake et al., 2023]. Hybrid systems combine the two, but still keep what is known about the world in rules or in weights. Gerundelle fixes neither the answers nor almost nothing. It fixes the mechanics of knowing: how ignorance is represented, how evidence is weighed and kept, and how understanding is revised. It proves their guarantees once, for every sequence, and leaves every contingent fact about the world to open variables funded by experience. An open variable is structural: it is defined by its place in the graph of what is already known, its type and its relations to known concepts, and those relations narrow what it can still

resolve to. Remove the relations and the open variable has no identity. Table 5 sets the three side by side.

This paper takes one step toward that goal, and its three parts follow the goal's three clauses. The relationships are defined explicitly, as the twelve layers of Gerundelle. Properties of their composition are proved. And the behavior that arises from evidence and the world is shown in use, in the tracks that run on the architecture (section 10).

For every layer we state guarantees and prove them in Lean 4 [de Moura and Ullrich, 2021] with Mathlib [The mathlib Community, 2020]. A guarantee is admitted only if it is an invariant the system's own design documents or tests already state; we do not invent promises the system does not make. The proofs hold for every possible input and every possible sequence of operations, not for the sequences someone tested. 301 theorems, 24 files, no unfinished proofs.

The guarantees are not isolated. Six areas of mathematics (each developed in its own field, rarely citing the others) turn out to be facets of one structure once stated about this architecture. Bridge theorems connect them: each states a result proved in one setting about another layer's own operators, ledger or data routing. Section 8 makes this precise.

1.4 A Cautionary Tale

Formalizing Gerundelle produced a lesson we believe is broadly relevant. To state the affect layer's guarantees, we translated the function that reads emotional phases off the ownership register into Lean, line by line. The curiosity reading would not translate: it tested a variable left over from an earlier loop, inside a loop over parked slots that can never be among the winners. In the case its specification describes (an unclaimed slot winning the ownership track), curiosity never formed; in another case it formed on the wrong slot; in a third it raised an error. The full test suite passed throughout.

Correcting the defect left the suite's pass count unchanged, and every seeded result that moved improved. A proof assistant caught what an 80-test suite could not: a function that was wrong on every input it was written for.

A second lesson came from the same layer. The guarantee we first proposed (that two minds alike in ownership feel alike) was false, as the code showed: curiosity reads the competition's winners and anger reads understanding. The true theorem is sharper: a proved map of which layer each phase reads (section 6.2).

1.5 A Running Example

To keep the twelve layers concrete, each one is illustrated by the same push followed through the system: a jar standing empty beside a spring, and a mind that pushes it. The jar charges; the mind perceives the change, attends to it, files a receipt, claims territory, and feels joy. Each step names the theorem it illustrates. The full trace is in section 2.3.

1.6 Contributions

1. The Gerundelle architecture: twelve layers from world physics to narrative selection.
2. Machine-checked guarantees for every layer, each derived from an invariant the system already states.
3. Bridge theorems connecting guarantees across layers: stand-alone results stated about the architecture's own operators, ledger and data routing, connecting perception to provenance, provenance to ownership, planning and selection, and perception to affect.

4. The factorization of perception and affect as one universal property at two levels, with a proof of which emotions break it and why.
5. Conservation of potential in the world, previously checked only by a run-time assertion, proved for every sequence of moves.
6. A defect in the reference implementation, found by formalization and corrected.
7. An explicit account of coverage: what each layer's model includes and leaves out.

2 Background

2.1 The SOV Algebra

SOV stands for *Structured Open Variables*: a graph of concepts connected by relations (caused by, conditions, co-occurs, role), each concept either open (an unknown: more than one resolution still feasible) or filled (known: exactly one remaining). Evidence narrows open concepts toward closure; contradiction reopens them. A mind's ignorance is not a blank but a shape: which concepts are open, what each still admits, and how the relations between them have narrowed each. An open slot is an information gap in Loewenstein's sense [Loewenstein, 1994], given geometry: typed, funded to a specific degree, and surrounded by specific known relations.

The algebra has two parts: the graph of what the mind currently understands, and the provenance record of how each piece was acquired.

Graph and provenance. The *graph* tracks which concepts exist, which slots are open or filled, and which relations connect them. The concept *jar-charged* has an open caused-by slot: what made it charge? The mind tries pushing the jar into the spring and the charge comes. That push fills the caused-by slot, linking *jar-charged* to *push* by a funded edge. A slot is *known* when exactly one resolution remains feasible; until then it is an unknown, exerting pull on the mind's attention.

The *provenance record* is a ledger: an append-only record of every receipt and where it came from. The push that funded the caused-by slot is a lived receipt (weight 1). Being told that pushing charges the jar is a told receipt (weight c_0). Imagining it funds nothing (weight 0). The ledger only grows. Nothing can be erased. Incremental update is a consequence of the algebra, not a feature bolted on: the ledger is append-only, so operators add entries and never remove them, and even a correction (a retraction or an eviction) is a new entry.

Formally, a *web* $w = (geom, ledger)$ pairs the graph part (a geometry of feasible sets and relations between slots) with the provenance part (the ledger of funded evidence). The geometry is a graph: concepts are nodes, relations between them are edges, and each edge narrows what its endpoints still admit. Each slot $i \in S$ carries a feasible set $F_i \subseteq R$; shapes are ordered by inclusion of feasible sets and met by intersection; a slot is known when $|F_i| = 1$. The ledger logs events, each citing the earlier entries that support it.

What the operators do. Everything the algebra can do to a web is done by a named operator. Each operator has two parts: what it does to the graph (which resolutions it rules out, which slots it creates or merges), and what it writes in the provenance record (what evidence it records). Table 1 lists the operators that appear in the theorems; the full set of seventeen is defined in `Operator.lean`.

A structural rule holds by construction: an operator's graph move is typed so that it cannot read the provenance record. The graph depends only on the evidence's structure, never on who filed it or when. The provenance record, by contrast, can read the graph: Fit checks whether a datum is feasible before filing it.

Operator	What it does to the graph	What it writes in the provenance record
Fit i x	nothing	a lived receipt on slot i if x is feasible, else to the unassigned pool
Constrain i j	narrows slot i's resolutions to those also feasible for j	a note that the narrowing happened
Exclude i j	records that slots i and j resolve differently	a note that they differ
Relabel i n σ	creates a new slot n with the old slot's resolutions translated	nothing
Differentiate	splits a slot's resolutions between two new slots	divides the receipts between them
Unify i j	merges two slots to the resolutions they share	pools j's receipts into i
Attest	nothing	imports another agent's receipts, at a discount
Archaize i	nothing	writes off slot i's lived evidence
Posit, Pose	nothing (Posit seeds the imagination register; Pose sends a slot's shape to another agent)	nothing

Table 1: The operators that appear in the theorems (ten of seventeen). Column headers name the two parts every operator has: what it does to the graph, and what it writes in the provenance record. The remaining seven (Individuate, Retract, Compose, Abstract, Bind, Quote, Anneal) are defined in `Operator.lean`.

Formally, each operator is a pair of a base move (on the geometry) and a fiber move (on the ledger):

$$op = (\mathit{base} : \text{GeomWeb} \to \text{GeomWeb},\ \mathit{fiber} : \text{Web} \to \text{List FiberEvent}),$$
$$op.\text{apply}\ w = (\mathit{base}\ w.\mathit{geom},\ w.\mathit{ledger} +\!\!+ \mathit{fiber}\ w).$$

The imagination register. A mind can imagine filling a slot without committing to it. The imagination register is a copy of the graph with no provenance record attached (a scratchpad where the mind tries things that don't count). Anything the mind knows about the graph (which resolutions are feasible, which relations hold) can be copied into the register. But the register carries no receipts: nothing imagined there can become evidence.

The inbound path exists: forget the provenance record and you have the register. The outbound path does not: there is no function that reconstructs funding from imagination. This asymmetry is proved (`no_outbound`), and the non-interference theorem (`firewall`) says that nothing done in the register ever reaches the funded web. The mind's funded understanding is untouched by anything it imagines.

Evidence and bids. Each piece of evidence on a slot is a trial (an attempt that either worked or did not) carrying a provenance: *lived* (the mind did it and saw the result; weight 1), *told* (someone said so; weight $c_0 \in [0, 1]$, equal to $\frac{1}{2}$ at perfect calibration), or *imagined* (the mind thought about it; weight 0).

How strongly an agent bids for a slot is computed from its trials: the ratio of successes to

Term	Meaning
gerund	What an object is doing (*resting*, *charged*, *welling*); the world's state is a gerund per object, not coordinates.
molecule	A candidate thought, with an activation, tags (what it is) and open sites (what it needs filled). "Molecule" because thoughts bond to each other the way chemical molecules do: by matching sites.
winners	Each tick, molecules bid for a fixed number of slots per level; the winners compete at the next level, and the last level's winners are what the mind attends to.
box, key	A thought that settles leaves the active pool and is stored as a sealed box, keyed by the concepts that resolved it. If those concepts reappear, the box reopens.
claim, take	A push on a region of the world is a claim; it *takes* if the predicted change comes.
ownership slot	An open question in the territory register about how the agent holds a region: *unclaimed* (never tried), *control* (tried, not yet taking), *contested* (another force acts there too), or *lost* (was taking, no longer).
phase	An emotional phase (frustration, curiosity, joy, ...), read off the territory register and anchored on what it is about.
want	A wanted change, held as a control slot the agent opens itself.

Table 2: Core vocabulary used across layers.

attempts, smoothed toward a base rate b by κ pseudo-counts:

$$\text{bid} = \frac{\text{took} + \kappa b}{\text{tried} + \kappa}, \tag{1}$$

where tried and took are the weighted counts. An agent with ten lived successes in twelve tries bids high. An agent with one told success bids low: the told weight halves the count. An agent whose only evidence is imagined bids at the base rate, because imagined trials add zero to both counts.

An auction awards an open slot to the highest bidder once its funding reaches a threshold M_0. A contradiction from the world reopens it.

2.2 Core Vocabulary

Table 2 defines the terms used across several layers. Terms used in a single layer, such as *warmth* in Memory or *meta molecule* in Self-awareness, are introduced where they are used.

2.3 A Running Example

One push, followed through every layer that touches it. This is the jar from the introduction. Each step says what happens in plain terms, then names the theorem that guarantees it in parentheses.

1. **World.** A jar stands *empty* beside a spring that is *welling*. The agent pushes the jar into the spring. The world draws potential from the spring's field into the jar's store, and the jar's gerund flips to *charged*. The total potential in the world is unchanged. *(Guarantee: `total_runMoves`: potential is conserved across any sequence of moves.)*
2. **Perception.** The agent perceives the fact *jar_charged*, not a number. The agent's world (its *Umwelt*) is the world simplified by what the agent's transforms cannot tell apart. Evidence is filed into this simplified world, not the raw one. *(Guarantee: `fit_factors_through_umwelt`: evidence routes through the agent's Umwelt.)*
3. **Provenance.** The datum is feasible for the push's slot, so Fit writes a lived receipt there. The slot's lived funding rises by one. Nothing the mind imagined about the jar could have

increased that number. *(Guarantees: `firewall`: imagination cannot reach the funded web; `replay_mass_le`: lived mass grows only through Fit.)*

4. **Competition.** The push's molecule bids for a slot in working memory. It wins only by outbidding every other molecule that wanted the same slot. *(Guarantee: `winner_outbids`.)*
5. **Ownership.** The claim took: the agent pushed the jar and the change it predicted came. The control slot on the jar, opened by the agent's want, closes with a lived take.
6. **Affect.** An ownership slot closed because a claim took. Joy forms, anchored on the jar. *(Guarantee: `mem_graph_joy`: joy forms on a closed claim.)* Had the push failed twice running, frustration would have formed instead *(`mem_graph_frustration`)*.
7. **Selection.** The author tells the push and its outcome as one sentence, citing the sentence that told the want. The citation points strictly back: no sentence cites a later one. *(Guarantee: `sourced_rebuild`: the told story is well-founded.)*

2.4 Lean and Mathlib

Lean 4 is a *proof assistant*: a program that checks mathematical proofs the way a compiler checks code [de Moura and Ullrich, 2021]. You write a theorem and its proof. If the program accepts it, the theorem holds: not because you tested it, but because the logic was checked step by step. Mathlib is Lean's library of formalized mathematics [The mathlib Community, 2020], providing standard definitions and results the development builds on.

The development builds with one command (`lake build`) and contains no unfinished proofs: no `sorry` ("trust me"), no `admit` ("skip this"), no `axiom` ("assume this"). Every theorem depends only on Lean's three standard logical axioms, the same ones all of Mathlib uses. This can be checked programmatically (`collectAxioms`).

3 Method

3.1 The Twelve Layers

Gerundelle groups its twelve layers into four, by the kind of question each group answers (fig. 1).

- **The machine.** *Can the mind see, attend, and tell evidence from hearsay?* Four layers: World (a physics that conserves), Perception (what the agent can distinguish), Competition (scarce attention slots), Provenance (the difference between lived, told and imagined). The guarantees in this group are about conservation and invariance: things that don't change no matter what the mind does.
- **The cognition.** *Can the mind remember, understand why, and notice when it is confused?* Three layers: Memory (thoughts that settle and reopen), Understanding (open questions with typed slots), Self-awareness (a mind that detects its own loops and confusion). The guarantees are about local structure: what holds in the neighborhood of one slot, regardless of what the rest of the web looks like.
- **The will.** *Can the mind care and feel?* Two layers: Ownership (territory: what the agent holds, contests and loses) and Affect (emotions read off the territory register). The guarantees are about dynamics that depend on order: ownership is non-commutative (the same events in two orders can leave a slot held in one and open in the other), and emotions track which way the territory moved.
- **The reach.** *Can the mind plan far ahead, learn from others, and tell what happened?* Three layers: Planning (sub-wants, constraint convergence, the curriculum), Social (attestation, trust, the discount), Selection (what is worth telling, and why). The guarantees are about

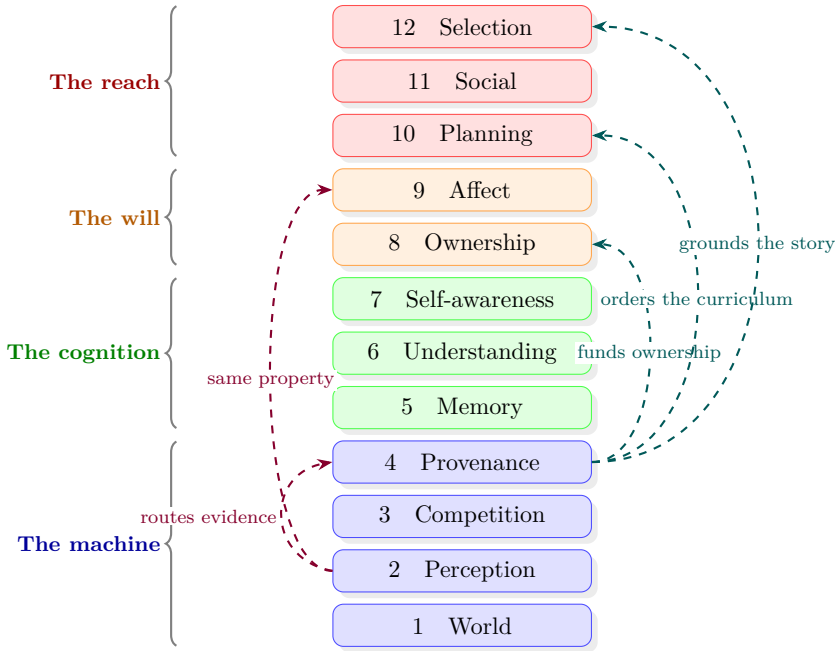


Figure 1: The twelve layers of GERUNDELLE, in four groups. Dashed arrows are *bridge theorems*: results proved in one setting stated about another layer's own operators, ledger or data routing. On the left, perception and affect are both instances of factorization through a view, and Fit routes evidence to the provenance ledger by the Umwelt. On the right, the provenance spine: the history of evidence funds ownership, orders the curriculum for planning, and grounds every sentence of a told story.

paths: every curriculum is a shortest path, every told story is well-founded, every attestation carries its provenance chain.

Each group's technical name (*epistemic conservation and invariance*, *local epistemic geometry*, *non-commutative territorial dynamics*, *geodesic agency*) describes the kind of mathematics its guarantees use. These names appear in the section headings and are not used elsewhere.

3.2 From Implementation to Model

The reference implementation is about 7,900 non-blank lines of Python in ten files. The Lean development is about 4,350 lines.

Each Lean model is derived from a named Python function by a fixed recipe:

1. The mutable state the function reads (dictionaries, counters, flags) becomes an explicit record: a list of named fields, each with a type. Nothing is hidden.
2. The function itself becomes a pure function of that record: same input, same output, every time. No side effects.
3. Loops that collect results become set comprehensions. Dictionary lookups with defaults become fields. An update in place becomes a function from one record to another.

Section C shows the Python and the Lean side by side for one function (the curiosity reading that turned out to have a bug).

A guarantee is admitted only if it is an invariant the system's own design documents or tests already state. We do not invent promises the system does not make.

Layer	Lean files	Source in the reference implementation
1 World	`World`	`civilization/potential_world.py`
2 Perception	`Erlangen`	design: `sov_geometry` §1–2
3 Competition	`Competition`	`Mind.think`, `Mind.resolve`
4 Provenance	`Ledger`, `Provenance`, `Firewall`, `Market`, `Crypto`	SOV spec; `Territory` (`Crypto`: negative result on a related secrecy conjecture)
5 Memory	`Memory`, `Quotient`	`reopen`, `contradict`, `tell`
6 Understanding	`Geometry`, `Slot`, `Operator`, `Holonomy`, `Confluence`	SOV spec
7 Self-awareness	`Meta`, `Competition`	`class Meta`
8 Ownership	`Market`	`Territory.pull`, `strength`
9 Affect	`Affect`	`Phases.graph`
10 Planning	`Planning`, `Pacing`	`sov_agent.py`; `Mind.support`
11 Social	`Discount`, `Monad`, `Integrity`	`attest`, `bill_attest_outcome`
12 Selection	`Selection`	`fiction/story.py`, `rung7.py`
All	`Bridges`	

Table 3: Lean files by layer and the Python code each model is derived from.

What this proves, and what it does not. A theorem holds of its definitions. The claim is that the guarantees hold of these models (the Lean functions derived from the Python code). Whether a model faithfully captures the code is argued, not proved: the Lean function could miss a case the Python function handles, or handle a case the Python function doesn't. We address this as follows:

- Section B lists **every hypothesis** that stands in for a property of the code: what it assumes, and how the assumption was established (by reading the code or by argument).
- Section A lists **what each layer's model leaves out**: which parts of the code are not modeled, and why (too implementation-specific, or beyond the current development's scope).

The reader who wants to know "did you prove the code correct?" should read these appendices. The answer is: we proved guarantees about models derived from the code, and we listed every gap between the models and the code. The gaps are there to be closed, and closing any one of them would strengthen the corresponding guarantee without changing the others.

3.3 How to Read the Theorems

Each layer's guarantees are presented in a fixed pattern:

1. A **Theorem** block states the guarantee in plain terms that match the Lean statement. The Lean name appears in `typewriter` so the reader can find it in the development. For the flagship results, the Lean signature is given in full.
2. A **Reading** block says what the guarantee means for the mind: what would go wrong if it failed, and why it matters.
3. A **Prior work** block credits the classical result the guarantee instantiates: the mathematics is usually known; what is new is that it holds of this architecture, machine-checked.

The formal reader can skip the Readings. The general reader can skip the Lean signatures. Both follow the same argument.

4 The Machine: Epistemic Conservation and Invariance

The first four layers answer: can the mind see, attend, and tell evidence from hearsay? Their guarantees are about things that hold no matter what happens: conservation laws and invariance properties that every sequence of operations preserves.

4.1 Layer 1: World

The world keeps one physical quantity (potential) in several places: a spring's field, a jar's store, what each object embodies, what each object carries in motion. Every change the world makes moves an amount from one place to another: drawing fills the jar from the spring, spilling empties it, forming spends it to make something, launching converts it to motion. Nothing is created or destroyed.

Theorem 4.1 (Conservation of potential)**.** *Lean:* `total_runMoves`, `nonneg_runMoves`

No sequence of moves changes the total potential. If every move takes at most what its source holds, every place stays nonnegative.

Reading. The system's design document stated this as "a theorem to prove in Lean." The implementation checked it with an assertion after every tick: if the total changed, the program crashed. Conservation held in every run reported. But "held in every run" is not "cannot fail." The theorem is: it cannot. For any sequence of moves, in any world that follows the rules, the total is unchanged.

4.2 Layer 2: Perception

Two agents in the same world may perceive it differently. One distinguishes the jar's charge level (empty, charged, brimming); another sees only empty or not-empty. The second agent lives in a coarser world: fewer distinctions, fewer things to attend to, fewer questions to ask. What an agent *cannot distinguish* determines what it perceives.

Formally: an agent's transforms form a group T acting on observations O. The agent's *Umwelt* (its perceived world) is the quotient O/T: things the agent cannot tell apart are one point. A reading f of observations is *invariant* if every transform preserves it.

Theorem 4.2 (Perception is determined by invariance)**.** *Lean:* `invariant_iff_factorsThrough`, `InvariantUnder.mono`

A reading of the world is something the agent can perceive exactly when it depends only on what the agent can distinguish. More precisely: a reading is invariant under the agent's transforms if and only if it factors through the map to the Umwelt. Enlarging the group of transforms (making more things indistinguishable) can only remove invariants; the agent sees less.

```
theorem invariant_iff_factorsThrough (f : O → X) :
    Invariant T f ↔ Function.FactorsThrough f
      (Quotient.mk (MulAction.orbitRel T O))
```

Theorem 4.3 (Evidence routes through the Umwelt)**.** *Lean:* `fit_factors_through_umwelt`

When the slot's feasible set is invariant under the agent's transforms, what Fit writes to the ledger for a datum depends only on the datum's point in the Umwelt, not on which raw observation produced it. Two observations the agent cannot tell apart produce the same evidence.

```
theorem fit_factors_through_umwelt
    (hF : ∀ (t : T) (x : R), t • x ∈ F i ↔ x ∈ F i) :
    Function.FactorsThrough (fun x ↦ (Operator.fit i x c e).fiber w)
      (Quotient.mk (MulAction.orbitRel T R))
```

Reading. The first theorem says: the mind cannot perceive what its own transforms cannot tell apart. If you can't distinguish red from green, no amount of attention will let you read a red-green traffic light. The invariants available to you are set by the distinctions you can make, not by the distinctions the world contains.

The second theorem connects perception to evidence. It says: two things the agent sees as the same produce the same evidence on the ledger. The mind does not file one observation in one slot and an indistinguishable observation in another. Evidence respects perception.

Together they mean: the agent's funded understanding is about its own Umwelt, not about the raw world. An agent with coarse perception builds coarse understanding. An agent with fine perception builds fine understanding. Neither is wrong; they live in different worlds.

Prior work. The first theorem is the factorization of invariant functions through the orbit space [Lehmann and Romano, 2005, Bronstein et al., 2021], the principle behind equivariant networks [Cohen and Welling, 2016]. "More transforms, fewer invariants" is Klein's hierarchy [Klein, 1893]. Reading O/T as an Umwelt is our interpretation of von Uexküll [von Uexküll, 2010], following Cassirer [1944]. The second theorem connects perception to the ledger; this bridge is new.

4.3 Layer 3: Competition

Each tick, every molecule in the working pool bids for a fixed number of slots per level. The winners advance; the losers stay behind. Only the top level's winners are what the mind attends to. If seven molecules bid for four slots, three of them lose, no matter how important they are.

This is attention by construction, not by a learned allocation. The guarantees say:

- At most k molecules win a level (`winners_length_le`).
- A molecule that did not win has a key no higher than any winner's (`winner_outbids`).
- A molecule reaches level $L{+}1$ only by winning level L (`tower_nested`).

Not everything that shapes thought competes for a slot. A *bond* (a learned link between two kinds of thing) is in superposition each tick: it can act as an *edge*, quietly connecting two molecules without taking a slot, or it can compete as a molecule in its own right to be examined and questioned. Most of the time a recognition does its work as an edge, below the competition. It reaches the top only when something goes wrong and the mind needs to examine the link itself.

Reading. Competition is what makes attention scarce, and scarce attention is what makes the market work. If every molecule could be attended to at once, there would be no tournament, no losers, and no pressure to bid accurately. A thought that wins does so by earning its place against everything else the mind could have attended to. The bond mechanism is why the mind can hold learned links ("she only takes calls on sunny days") without using a conscious slot for each one, until one of them is questioned.

Prior work. The tournament borrows from Global Workspace Theory [Baars, 1988] its limited-capacity competition: many specialized processes compete for a stage that only a few can occupy, with all-or-nothing access [Dehaene, 2014]. It does not borrow the theory's broadcast, in which the winners' contents are sent to every process, and no guarantee here concerns one. Gerundelle arrives at limited-capacity competition for engineering reasons, to avoid lock-in and to scale, and makes no claim of neural equivalence; the convergence is evidence that the design region is good. The slot budgets (7, 5 and 4 per level) are anchored in working-memory capacity [Miller, 1956, Cowan, 2001]. The Conscious Turing Machine formalizes global-workspace competition [Blum and Blum, 2022]; the guarantees here are the scarcity properties of such a tournament, machine-checked.

Two terms of a molecule's bid have their own lineage. Each open site adds a constant tension, so an incomplete thought keeps pulling until it is closed: a mechanism for the Zeigarnik effect

[Zeigarnik, 1927]. And the bid looks forward, carrying the surprise a molecule could still reduce, the novelty term of the free-energy principle [Friston, 2010], which the design adopted to cure the tournament's lock-in. Neither is a claim to implement active inference.

4.4 Layer 4: Provenance

Layer 4 is the paper's spine (section 8). Every other layer depends on what this one guarantees: that evidence is trustworthy, that imagination cannot corrupt it, and that every receipt traces back to the world.

The imagination firewall. A mind can imagine, dream, hypothesize and plan. None of it touches the funded web. The firewall theorem says: run any interleaving of real operations, imagination-register moves and Posit seeds. The funded web that comes out is the result of the real operations alone. Everything imagined is invisible to it.

Theorem 4.4 (The imagination firewall)**.** *Lean:* `firewall`

```
theorem firewall (sops : List SysOp) (w : Web) (h : ImagWeb) :
    (SysOp.run sops (w, h)).1 =
      Operator.run (sops.filterMap SysOp.funded?) w
```

Run any interleaving of funded operators, register moves and Posit seeds: the resulting funded web is the result of running the funded operators alone.

Evidence is what it says it is. Three guarantees about the provenance of evidence, stated separately because each protects against a different threat:

Theorem 4.5 (Imagined evidence is inert)**.** *Lean:* `bid_filter_imagined`, `run_owner_filter_imagined`

Removing every imagined trial from any event sequence changes no bid and no owner. A mind that dreams, hypothesizes or speculates cannot change what it owns or what it knows by doing so.

Theorem 4.6 (Told evidence cannot be promoted)**.** *Lean:* `took_told_le`

Told-only evidence funds at most c_0 per trial, whatever the processing. No sequence of operations can turn a told receipt into a lived one.

Theorem 4.7 (Lived evidence comes only from the world)**.** *Lean:* `replay_mass_le`

Replaying events none of which is a lived accrual (a Fit) does not increase lived mass. The only way to create lived evidence is to interact with the world.

Theorem 4.8 (No outbound path from imagination)**.** *Lean:* `no_outbound`

Every function from imagination registers to funded webs misreports lived mass on some web with the given geometry. The asymmetry is structural: forgetting funding is always possible; reconstructing it is impossible.

Self-citation and grounding.

Theorem 4.9 (Self-citation is legal but ungrounded)**.** *Lean:* `wellFounded_cites`, `not_transGen_self`, `not_grounded_of_selfCited`

In a log where every entry cites only earlier entries, citation is well-founded and no entry is in its own chain. A non-root entry citing only itself or later entries is never grounded. Circular reasoning is not prohibited; it is inert. A self-citing entry can exist; it cannot fund anything.

Reading. Taken together, these theorems say: **provenance is conserved.** What was lived stays lived. What was told stays told. What was imagined funds nothing. No sequence of operations (no amount of thinking, dreaming, rehearsing or reprocessing) can upgrade the provenance of a piece of evidence.

This is what makes the market's bids meaningful. A bid rests on evidence whose source cannot be upgraded after the fact. And it is what makes the ownership register trustworthy: a slot held by lived evidence was held by actual interaction with the world, not by imagination that leaked into the ledger.

The funding gate (the precondition that a non-Fit operator must cite at least one earlier receipt to fire) is proved necessary (`not_grounded_of_unfunded`): without it, ungrounded entries appear. The gate is not a safety margin. It is a necessary condition for grounding.

Prior work. The firewall is non-interference in the purge formulation of Goguen and Meseguer [1982]: the imagination register is a low-integrity domain that does not interfere with the funded web, as in type-based non-interference [Volpano et al., 1996, Sabelfeld and Myers, 2003]. Mechanized non-interference at scale has precedent in operating-system verification [Murray et al., 2013].

Inert imagined evidence is semiring provenance with tags valued $1, c_0, 0$ [Green et al., 2007]. The told cap is fixed-weight discounting [Jøsang, 2016]. Lived mass is a Petri-net conservation invariant [Murata, 1989]. The map-back lemma is the view-complement fact [Bancilhon and Spyratos, 1981, Foster et al., 2007].

Self-citation is Kripke's ungrounded truth-teller [Kripke, 1975], here with well-foundedness by construction, as in well-founded semantics [Gelder et al., 1991]. The funding gate is the well-founded support condition of truth maintenance systems [Doyle, 1979, de Kleer, 1986].

Dreaming, in the sense of sleep's replay and recombination of experience [Walker and Stickgold, 2004] and of generative replay in machine learning [Shin et al., 2017], is the register's natural use: replayed and recombined episodes can suggest what might fill a slot. The firewall is what makes this safe. Nothing dreamed funds anything.

5 The Cognition: Local Epistemic Geometry

The next three layers answer: can the mind remember, understand why, and notice when it is confused? Their guarantees are mostly about local structure: what holds in the neighborhood of one slot, one memory, one meta-observation. The exception is Understanding, whose holonomy result characterizes when a global frame exists across all domains.

5.1 Layer 5: Memory

A mind that never forgets drowns in its own history. A mind that forgets everything starts over every tick. Memory is the mechanism between: thoughts that have been fully examined leave the active pool and are stored, and some of them come back.

A thought that closes is stored as a sealed *box*, indexed by the concepts (its *keys*) that resolved it. Think of sealed envelopes with keywords written on the outside. Time fades the ink: each key's *warmth* halves every h ticks it goes unfilled, and below a threshold the key is cold and no longer checked.

A box returns to the pool in three ways:

- **Resonance.** Enough of its warm keys match what the mind is currently attending to. The box opens gently: the context it was closed in has partly returned.

- **A searching thought finds it.** A thought with an open site looks for what would fill it, and finds it in a stored box, behind a warm key.
- **Contradiction.** The world produces an outcome the box's prediction said would not happen. The box opens urgently, with surprise raised by how well the concept was understood, demanding re-examination.

Three guarantees make this precise:

Theorem 5.1 (Forgetting is cooling, not deletion)**.** *Lean:* `warm_of_le, not_resonates_of_cold`

A key warm now was warm at every earlier tick. A box whose keys are all cold cannot resonate. Memories do not vanish; they become unreachable as their context cools.

Theorem 5.2 (Memory does not ruminate)**.** *Lean:* `not_reopens_after_resonating`

A box that resonated this tick does not reopen by resonance at the next. A memory brought back once does not immediately bring itself back again; something else must intervene.

Theorem 5.3 (Lived and told memories differ)**.** *Lean:* `told_reopens_iff`

A lived box is keyed by the tags that resolved it (the concepts the mind was working with when it closed). A moment that resembles it can bring it back by resonance. A told box arrives already closed, with no keys of one's own, because the mind did not work it out. No resemblance can recall it. A told box reopens if and only if the world contradicts it.

Reading. The lived/told distinction is the memory layer's version of provenance conservation. What you worked out yourself has warming keys and can be gently recalled by a similar context. What you were told has no keys and can only be shattered by the world saying otherwise. "You don't own what you haven't worked out" is not a slogan. It is a structural property of how the box was sealed: no keys, no resonance path, only contradiction.

This is why a told belief can persist for a lifetime unchallenged; its keys never warm, so it is never re-examined by resemblance. It reopens when the world contradicts it, or when it is worked out: decomposed and recognised through live keys of one's own. Until then it sits sealed, without the resonance path that would revise it gradually.

Additional guarantees. In the ledger's history, a lived accrual and an eviction on the same slot do not commute (`accrue_evict_not_comm`): evicting after a lived receipt writes it off, while evicting before leaves it untouched. The trace quotient, which swaps only state-preserving events, respects ledger state (`SameTrace.replay_eq`) and is the least congruence identifying logs of such events that contain the same events (`SameTrace.minimal`). Affective phases that fade with their question still open are stored open and reopen when their subject returns (the design's "Pandora" case); this is not yet modelled (section A).

Prior work. Contradiction forcing a sealed box open is Kuhn's cycle of anomaly, crisis and revision [Kuhn, 1962] at the scale of a single belief: `told_reopens_iff` says a told belief opens only that way. That a memory does not ruminate (`not_reopens_after_resonating`) answers the clinical picture of rumination [Nolen-Hoeksema, 1991] with a mechanism: a box that keeps resonating is held closed until it goes out of resonance.

5.2 Layer 6: Understanding

When a mind learns something in one domain, can it carry that understanding to another? A carpenter who understands wood grain encounters a new kind of wood. Does the understanding transfer? Or does it distort: carry a distinction that doesn't apply, or lose one that does?

Transfer is modelled as *transport*: carrying a slot's geometry from one domain to another. Write τ_{ij} for the transport from domain i to domain j. If you transport from i to j and then from j to k and then from k back to i, you have gone around a triangle. The *holonomy* of that triangle is what changed in the round trip. If nothing changed, the triangle is *flat*: the understanding transferred perfectly. If something changed, the understanding was distorted by the path it took.

Theorem 5.4 (Flat iff trivializable)**.** *Lean:* `flat_iff_trivializable`

For invertible transports given on every ordered pair of domains, every triangle has zero holonomy (perfect transfer) if and only if there exist global coordinates (a single frame of reference that works in every domain). Where global coordinates exist, the path doesn't matter: going the long way around gives the same result as going directly.

$$(\forall i,\ \tau_{ii} = 1) \wedge (\forall i\, j\, k,\ \mathrm{hol}(i,j,k) = 1) \iff \exists\, \varphi : \iota \to \mathrm{Perm}(G),\ \forall i\, j,\ \tau_{ij} = \varphi_j\, \varphi_i^{-1}.$$

Theorem 5.5 (Relabel makes understanding local)**.** *Lean:* `loop_perm`, `relabel_path_dependent`, `relabel_accumulates`, `relabel_not_meets`

Transport by meets alone (Constrain) is path-independent: it doesn't matter which order you apply the constraints. But Relabel (translating the vocabulary from one domain to another) makes transport path-dependent. Going around a triangle with a Relabel step can change the understanding, and Relabel cannot be decomposed into meet steps: it is a genuinely new operation.

Reading. Where vocabulary translation between domains is needed, there is no global frame. Structure is inherently local, and it is the algebra's own operators that make it so (`relabel_is_transport`, `constrain_is_transport`).

Constrain and Exclude are confluent (`p66_constrain_exclude`): the same constraints applied in any order reach the same geometry. Fit does not commute with Constrain or Archaize (`fit_constrain_not_perm`, `fit_archaize_order`): the order of evidence and narrowing matters for the ledger, even when it doesn't matter for the geometry.

A discovery has an epistemic order; the steps cannot be rearranged and produce the same result. The holonomy theorem proves this for the algebra's own operators: transport by Relabel is path-dependent, and that path-dependence is not removable. Going around the triangle the long way is not the same as going directly, because Relabel is not a meet step.

Prior work. The flat-iff-trivializable theorem, on the complete graph of domains, is the coboundary criterion for a non-abelian 1-cocycle [Giraud, 1971], the discrete analogue of a pure gauge in lattice gauge theory [Wilson, 1974], known in vision as cycle consistency or synchronization [Singer, 2011, Pachauri et al., 2013, Huang and Guibas, 2013, Gao et al., 2021]. Local consistency without a global section is the subject of sheaf methods [Hansen and Ghrist, 2019, Abramsky and Brandenburger, 2011]. Relabel's underivability from meets is an instance of Howie [1966]: products of idempotent transformations are never non-identity permutations.

5.3 Layer 7: Self-awareness

A mind that cannot notice its own confusion stays confused. A mind that monitors its own processing at the cost of the processing itself notices everything and does nothing. Self-awareness is the layer that negotiates between these.

The mind forms *meta molecules*: molecules about another molecule's state. "I am stuck," "I am looping," "this thought gives two different outcomes and I don't know why." Meta molecules compete; they do so only among themselves, in their own dedicated slots, so they never crowd out regular thoughts.

Two hazards govern this layer, each a design rule guarding against a failure mode the design documents name:

The mirror trap. A meta molecule that refreshes itself by re-detecting the state it was about would consume a slot, contribute nothing, and resist removal because the state it reported was still true: rent without yield, a mirror that reflects without acting. The design guards against this: every meta molecule carries a *prediction* about its subject, with a *deadline*. It lives by being right about the future, not by re-observing the present. A prediction confirmed extends the deadline. A prediction refuted kills the molecule. No re-detection extends its life.

Theorem 5.6 (The mirror trap is closed)**.** *Lean:* `detect_standing`, `deadline_changes_only_by_confirmation`, `refuted_dropped`, `check_before_deadline`

Detecting a standing state again does not extend a meta molecule's life. The deadline moves only when the prediction is confirmed. A refuted molecule is dropped. None outlives its deadline undecided.

One-way aboutness. A meta molecule about a thought should not inflate the thought it is about. Without a guard, "I notice this" could boost the thing noticed, creating a feedback loop: noticing makes the thought louder, which makes the noticing more relevant, which makes the thought louder. The design rules this out.

Theorem 5.7 (Meta cannot inflate base)**.** *Lean:* `base_filter_meta`, `partner_not_meta`, `bid_ignores_meta`

Meta molecules compete in separate slots and never displace a thought. A thought never binds to a meta molecule. Adding meta molecules never changes a thought's bid. Meta operations such as parking or splitting do act on their subject, but the bid (the measure of competitive strength) is unaffected.

Reading. Self-awareness is not free. The meta molecule must *win* a slot to act; it must outcompete other meta molecules, not regular thoughts. The capacity to notice confusion exists whether or not confusion is noticed: the meta molecule is in the pool, but it might lose to a more urgent meta-observation. This is why you can be stuck without knowing you are stuck, and why the realization "I'm going in circles" surfaces only when the meta molecule finally wins.

The aboutness guarantee protects against the architectural version of rumination: a mind that notices its own anxiety and is made more anxious by noticing. The one-way rule keeps the observation channel silent: the camera doesn't disturb the wildlife.

Additional guarantees. A meta molecule binds only to its subject (`meta_partner_is_subject`), and the tower of meta levels is bounded in depth (`emitDepth_le`). Operations such as parking or splitting fire only for a meta molecule among the top winners (`operated_mem_top`), and kinds with an operation take those slots first (`operable_win_first`). A want is never parked (`not_goal_of_parkable`). When a thought's outcomes go two ways, splitting it on a boundary partitions its outcomes; mending a wrong split restores them, keeping any outcomes gathered since (`mem_split_iff`, `unify_differentiate`, `unify_keeps_later`).

Prior work. Splitting a conflated thought is Piaget's accommodation [Piaget, 1952]: a schema revised because the world will not fit it. The conflations it acts on are the bimodal outcomes Carey documents in children's conceptual change [Carey, 1985], and mending a wrong split is the revision run the other way. A meta layer that notices and operates on the system's own representations is representational redescription in the sense of Karmiloff-Smith [1992].

6 The Will: Non-commutative Territorial Dynamics

The previous layers explain how a mind perceives, attends, remembers and understands. None of them explains why the mind acts at all. Curiosity ranks questions without originating wants. Understanding fills slots without saying which slots matter. Something else is needed: a reason to push.

The answer is territory. An agent acts to hold what it perceives as its own.

6.1 Layer 8: Ownership

Territory is whatever the agent perceives as its own: a body, a place, an object, an idea, a relation, or a slot in another mind. Being on someone's mind, living rent free in their thoughts, is territory too: a relation in their understanding whose filler is the agent. The reference implementation models it as a slot another mind holds open for the agent (until she answers, the agent is not on her mind), and credits actions to it; the Lean development does not yet cover it, and the theorems below concern regions of the world.

For each region of the world, the territory register tracks how the agent perceives its holding, and opens an ownership slot where the boundary is incomplete. Three slot kinds govern how hard each region pulls on the mind's attention:

- **Unclaimed** (never tried): "What happens if I push here?" A spring the agent has never drawn from, an object it has never touched. It pulls at curiosity $\cdot\, \kappa/(\kappa + \text{other})$; less when something else already moves the region, because a region that moves on its own is harder to claim.
- **Control** (tried, not yet taking, or wanted): "What blocks it?" A lid the agent has pushed and that didn't move, a want that hasn't been met. It pulls harder the longer the agent has pursued it: $1 + \text{tried}/(\text{tried} + \kappa)$.
- **Lost** (held once, fallen back): "What took it?" A toy the agent played with and that was carried away. It pulls hardest of all: $1 + \text{held}$. The more was held, the more there is to lose. This is loss aversion as a structural property, not as a bias parameter.

A held region that something else also moves, or on which a claim has just failed, opens a *contested* slot. The affect layer reads it.

Two guarantees define the ownership layer's dynamics:

Theorem 6.1 (Funding is order-free; ownership is not)**.** *Lean:* `bid_perm`, `ownership_order_dependent`

A bid depends only on the multiset of trials; the order of the evidence doesn't matter for what the agent knows. But ownership depends on the sequence: the same three events (a lived take, a close, a contradiction) in two different orders leave a slot open in one case and held in the other.

Reading. This pins down exactly where the architecture's non-commutativity lives. What you know is the same regardless of the order you learned it in: bids are order-free. But who holds what depends on history: a slot taken and then contradicted ends open, while the same slot contradicted and then taken ends held. Evidence commutes. Territory doesn't. History matters for ownership the way it matters for a treaty: the last event, not the first, determines what holds.

Theorem 6.2 (The pull order, and its temperament inversion)**.** *Lean:* `pull_order`, `restless_inverts`

With curiosity at most 1, given that the lost slot has held at least once and the control slot has been tried: lost pulls harder than control, control harder than unclaimed. With curiosity at 2, an unclaimed region that nothing else moves outpulls every control slot.

Reading. The pull order IS temperament, with proved boundaries. In the default range (curiosity at most 1), lost slots pull harder than control slots, which pull harder than unclaimed ones: the mind attends to what it lost, then to what it is pursuing, then to what it hasn't tried. In the restless range (curiosity at 2), an unclaimed region that nothing else moves pulls harder than any control slot. The order is proved for curiosity at most 1, and its inversion at curiosity 2; the gap between is open. Different parameter settings produce different characters with proved structure, not accidental variation.

Prior work. The split between commutative aggregation and order-dependent revision is standard in belief revision [Alchourrón et al., 1985, Darwiche and Pearl, 1997]. In reinforcement learning, exploration is a bonus added to reward: a count-based novelty term [Strehl and Littman, 2008, Bellemare et al., 2016] or an intrinsic-reward weight [Pathak et al., 2017, Oudeyer et al., 2007]. Here it is not a bonus. An unclaimed region is an ownership slot competing for attention like any other, and how strongly it pulls is a matter of temperament. What it shares with artificial curiosity [Schmidhuber, 1991, Oudeyer et al., 2007] is the intuition that curiosity is about what could be learned, not a hand-coded reward.

That loss pulls hardest follows the revised ethology of the wolf pack: the "alpha" is not a dominant animal but the one with the most to lose [Mech, 1999]. Territory is about what is held, not about dominance. Ownership as the foundational motivation meets self-determination theory's three needs [Deci and Ryan, 1985, Ryan and Deci, 2000]: autonomy is control (the agent's push works), and competence is understanding funded by control slots.

The market is an auction for control in the lineage of Baum [1999] and Wellman [1993]; funding here is provenance-weighted evidence, not transferred money.

6.2 Layer 9: Affect

Emotions are not labels applied after the fact. They are states of the ownership register, read off its graph each tick. The agent does not decide to feel frustrated. The register says: a live slot has failed twice running. That IS frustration.

Each tick, the phases are read from a *snapshot*: the register's open and live slots, each region's history of failures, the claims that took, and a context of what lies outside the register (who won the competition, whom understanding blames).

Theorem 6.3 (The phases)**.** *Lean:* `mem_graph_frustration`*,* `anxiety_of_one_failure`*,* `mem_graph_joy`*,* `live_empty_of_calm`*,* `mem_graph_curiosity`

Each emotional phase has an exact formation condition:

- ***Frustration:*** *a live slot (control or contested by its own failure) with two or more failures running. The claim keeps failing.*
- ***Anxiety:*** *one failure on a contested slot. A held region under pressure.*
- ***Joy:*** *an instance closed because a claim took. The push worked.*
- ***Calm:*** *nothing is live. No open slot, no unfinished business.*
- ***Curiosity:*** *an unclaimed live slot won the ownership track. Something untried pulled hard enough to win attention.*

Theorem 6.4 (Which layer each phase reads)**.** *Lean:* `register_phases_factor`*,* `curiosity_not_factor`*,* `curiosity_factors`*,* `anger_not_factor`

```
def view (s : Snapshot) : Register × Memory := (s.reg, s.mem)
theorem register_phases_factor :
    FactorsThrough (reading registerKinds) view
theorem curiosity_not_factor :
```

```
    ¬ FactorsThrough (reading {.curiosity}) view
theorem curiosity_factors :
    FactorsThrough (reading {.curiosity}) (fun s ↦ (s.reg.live, s.ctx.top))
theorem anger_not_factor :
    ¬ FactorsThrough (reading {.anger}) view
```

Frustration, anxiety, sadness, joy, calm, gratitude and resignation all factor through the register and the previous tick: they depend only on what the agent holds, not on what it thinks or whom it blames.

Curiosity does not factor through the register alone. It also reads the competition's winners: curiosity depends on what won attention, not just on what is unclaimed.

Anger does not factor through the register alone either. It also reads understanding and the meta layer: anger is anger at someone*, and whom to blame is inferred from who was seen taking what was lost and, through understanding, from whatever keeps the state a failing claim depends on.*

Reading. The positive result and the two negative results together form a **map of how affect connects to the other layers**. Most emotions live in the ownership register alone. Curiosity reaches into competition. Anger reaches into understanding. The map is proved, and it is sharper than the theorem we first set out to prove.

Co-occurrence is not causation: getting the right emotion in the right place is not the same as knowing why it is there. The proved map is the difference. It does not just correlate phases with states; it identifies the exact conditions and the exact layers each phase reads, and it proves which phases break the factorization property and why.

The guarantee we first proposed was: two minds alike in ownership feel alike. It was false, as the code showed: curiosity reads the competition's winners, anger reads understanding. The true statement is the map above: which phases read which layers. This is the second lesson of the cautionary tale (section 1).

The mathematical structure is the same one that governs perception (Layer 2). `FactorsThrough` is the same property that, with the Umwelt's quotient map in place of `view`, says "a reading is something the agent can perceive exactly when it is invariant." Applied here, it says: "a phase is something the ownership register determines exactly when it factors through the register." Perception factors through the Umwelt; affect factors through the ownership register. Same property, two levels. Bridge theorem: section 8.

Anger is the instructive case for why the map matters. Blame can spread: each new thing that fills a blocked goal's "caused by" slot gets a copy of the anger, and those copies compete in the thoughts' own slots (the cascade the design calls a *cancer of the mind*). A grounded copy (one whose blame the records support) closes when its goal is met. A copy whose blame the records do not support escapes that closure and only fades. `anger_not_factor` is the formal trace of this: anger cannot be read off ownership alone because blame is not an ownership fact.

Prior work. Phases are appraisals made executable [Lazarus, 1991, Scherer, 2001]: each is a structured evaluation whose anchor is what is appraised and whose kind is the result. In the sense of Damasio [1994], they are not noise on reasoning but what lets the agent prioritize: they redirect which questions it pursues. The spread of ungrounded anger copies through the thoughts' slots is rumination [Nolen-Hoeksema, 1991] given a mechanism.

7 The Reach: Geodesic Agency

The last three layers answer: can the mind plan far ahead, learn from others, and tell what happened? Their guarantees are about paths: valid paths through a space of understanding, valid paths from

one mind to another, and valid paths through a story.

7.1 Layer 10: Planning

Sub-wants: when the goal is too far. The planner looks three steps ahead. A mind on the moon that wants to go home faces a seven-step chain: unmake the husk, draw from the pool, draw again, form the lantern, fuse it onto the skiff, fill the flask, launch. Seven steps. Three-step horizon. The goal is invisible from where the mind stands.

The solution: the mind sets a nearer goal (a *sub-want*) and keeps the distant want on a stack. "I need the pool glimmering before I can light the skiff, so I'll want that first." Each sub-want is within the planner's horizon. The mind reaches it, pops the stack, and the next sub-want takes over.

Theorem 7.1 (Sub-wants are bounded)**.** *Lean:* `inv_run, drops_le_card`

Along any run, the sub-want stack is at most four deep. No want appears on the stack twice. No sub-want once dropped is set again. Over finitely many facts, a run drops at most as many sub-wants as there are facts.

Reading. The stack is bounded and non-repeating: no want appears twice on the stack, and a dropped sub-want is never set again. A mind chasing a far goal cannot loop on the same sub-want or accumulate an unbounded stack; each step either advances or exhausts a distinct option.

Constraint convergence: finding the one that survives. The mind does not search for solutions. It searches for *constraints*; the solution is the one left standing when the constraints converge.

A hand reaching for a hot mug full of coffee: the body is hot (learned from a burn), the rim is chipped (learned from a cut), it must not tilt (learned from a spill), the bottle blocks one side (learned from a knock). Under those constraints, perhaps one grasp survives: the handle, from the right, fingers through.

Theorem 7.2 (JustOne)**.** *Lean:* `justOne`

When exactly one attempt meets every live constraint and each other attempt misses at least one, that attempt is the unique best supported.

Reading. The solution is not computed. It is *noticed*: the one attempt that meets every constraint wins the tournament by simultaneous support. The mind never searched the space of grasps. It searched the space of constraints, which is small, and the grasp was what was left.

Failures are constraint revelations, not negative signals. A failure that reveals a new constraint is worth more than a success that confirms what was already known: the success changes nothing about which attempt survives, while the failure shrinks the space the next attempt must occupy.

The curriculum is a geodesic. What can be learned in what order is a structured space. Each concept has prerequisites, and a valid curriculum visits them in order.

Theorem 7.3 (Every valid curriculum is a shortest path)**.** *Lean:* `card_learn, never_enters, reachable_of_downClosed`

A valid curriculum adds one concept per step, never enters a prerequisite cycle, and reaches every achievable state when prerequisites form a strict order. No valid path wastes a step.

Reading. The mind's understanding grows along the shortest path that respects its prerequisites. A prerequisite cycle is unreachable (proved, not assumed). The curriculum can't get stuck in a loop

and can't take a detour. Every achievable state of understanding can be reached by some valid curriculum.

The cost of search scales with ignorance, not with the search space. The more the mind understands (the more slots are funded, the more constraints are live), the fewer candidates survive to be tested. Understanding is what makes search unnecessary. That is what understanding is for.

Prior work. Valid curricula are the learning words of the ordinal knowledge space of the prerequisite relation [Doignon and Falmagne, 1985, Falmagne and Doignon, 2011, Cosyn and Uzun, 2009], the case with conjunctive prerequisites only. A concept on the fringe (unknown, with every prerequisite known) is what the learner is ready for next: Vygotsky's zone of proximal development [Vygotsky, 1978], made exact for this case.

7.2 Layer 11: Social

Minds exchange knowledge by Pose (sending a geometry to another mind) and Attest (importing another mind's receipts at a discount). The central question: when knowledge passes from one mind to another, what happens to its provenance?

Theorem 7.4 (Testimony stays testimony)**.** *Lean:* `lamarckian_ill_graded`

The arrow pairing Pose's geometry move with a lived accrual does not have Pose's grade: it is ill-typed in the graded monad. Knowledge passed between minds arrives as told, never as lived.

Reading. A parent who has learned by doing cannot pass that doing's provenance to a child. The child receives the content (what the parent learned) but not the experiential warrant. The child's receipt is told, at half weight, and must be tested to become the child's own.

Pose's grade is empty because of what Pose is: it shares structure, not experience. With that grade, no arrow that writes a lived event can carry it, so the Lamarckian pairing (inheriting the experiential warrant of acquired characteristics) is a type error in the graded monad, not a rule that could be relaxed without changing what Pose is. The provenance firewall extends across lifetimes, not just across the imagination boundary.

Tacit knowledge is only tacit when it lives in weights. When it lives in funded constraints with provenance, it is inspectable: the expertise and its justification are the same structure. A bond that fires below the competition's top level still carries the trials that funded it. Every constraint the mind acts on has a receipt.

Additional guarantees. The two-stage attestation discount stays in $[0, 1]$ at every stage of calibration (`discount_mem`), so the evidence guarantees of Layer 4 apply to testimony. Against an adversary that controls a signed log, an honest agent's viewed lived mass is at most what it signed (`honest_livedMass_le`); the adversary can only truncate (`accepted_take`).

Prior work. The grading is a graded monad in the sense of Katsumata [2014], presented as an effect predicate on a state-writer monad [Lucassen and Gifford, 1988, Fujii et al., 2016, Orchard and Petricek, 2014]. The Lamarckian distinction is from cultural evolution theory [Boyd and Richerson, 1985]; what is not transmitted is the experiential warrant [Kronfeldner, 2007].

7.3 Layer 12: Selection

A mind that acts in a world produces a history of beats: what it did, what happened, what it realized, what it felt. The story layer selects which beats to tell, in what order, and the fiction pipeline prints each one as a sentence with an intended structural skeleton. The determination

system reads each sentence back: a sentence passes only if a reading has the shape the construction intended.

Theorem 7.5 (A story is a provenance log)**.** *Lean:* `sourced_append`*,* `sourced_rebuild`*,* `scene_wellFounded`*,* `scene_not_transGen_self`

Each told sentence cites only earlier sentences. Every rebuilding pass preserves this. Dependence among sentences is well-founded, and no sentence depends on itself.

Theorem 7.6 (A deus ex machina scores zero)**.** *Lean:* `value_eq_zero_of_unforeshadowed`*,* `value_pos_iff`

The author's score is suspense held late times the rise of the ending's probability. An ending no likelier just before it became certain than at the start earns nothing: it came from nowhere. Value is positive exactly when there is both suspense (the reader didn't know) and foreshadowing (the text moved the reader toward it).

Reading. A story is the same kind of structure as evidence. Each sentence cites what came before it, and nothing cites the future. The ledger's well-foundedness theorem, proved once for evidence in Layer 4, is the same theorem that grounds a told story in Layer 12.

The deus ex machina theorem says: value requires both uncertainty and convergence: the same tension between suspense and foreshadowing that every storytelling manual prescribes, here proved as a product whose factors are both necessary.

Ownership is load-bearing for narrative arc: the tension in a told story comes from failing claims, and resolution comes from claims that took. The selection layer does not impose narrative shape on the history; the ownership register already has it. The author reads it off.

8 Analysis: The Unification

The guarantees above were proved layer by layer. Several were first stated over generic structures: groups acting on sets, permutations between domains, trials with weights, prerequisites with an ordering. Bridge theorems then stated each result about the architecture's own operators, ledger and data routing. The result: six areas of mathematics that rarely cite one another in the literature describe the same system.

8.1 Bridge Theorems

A bridge theorem takes a result proved generically and states it about the architecture's own operators, ledger and data routing. Table 4 lists them.

8.2 Perception and Affect Are One Property

The last row of the table is the clearest instance of the paper's thesis. Perception and affect share one mathematical structure: factorization through a view.

For perception: a reading of the world is something the agent can perceive exactly when it depends only on what the agent can distinguish. The view is the Umwelt.

For affect: an emotional phase is something the ownership register determines exactly when it depends only on what the register holds. The view is the register.

Neither result is derived from the other. What is proved is that both are instances of one universal property. And the negative results (curiosity doesn't factor, anger doesn't factor) tell you exactly where the property breaks and which other layer each failing phase reaches into.

Perception factors through the Umwelt; the register phases factor through the ownership register. Same structure. Two levels.

Layers joined	What the bridge says	Lean
Understanding	Relabel and Constrain are the transport steps of the holonomy theory: the locality results are about the algebra's own operators, not abstract permutations.	`relabel_is_transport`, `constrain_is_transport`
Provenance → Ownership	A slot's market funding is its lived accruals at weight 1 plus its attested imports at c_0; imagined content yields no trial. The provenance layer's evidence is the ownership layer's currency.	`took_trials`
Provenance → Planning	Reading the ledger in order is a valid curriculum. Every valid curriculum over it is a geodesic. The history of evidence is the mind's own textbook, in the right order.	`reading_in_order_valid`, `ledger_curriculum_geodesic`
Provenance → Selection	A story is a provenance log: the well-foundedness theorem for evidence is the same theorem that grounds sentences.	`scene_wellFounded`
Perception → Provenance	When a slot's feasible set is invariant under the agent's transforms, Fit routes evidence by the Umwelt: two observations the agent can't distinguish produce the same filing.	`fit_factors_through_umwelt`
Perception → Affect	Perception factors through the Umwelt; the register phases factor through the ownership register. Same property, two levels.	`invariant_iff_factorsThrough`, `affect_factorsThrough`

Table 4: Bridge theorems. Each joins two layers by stating a stand-alone mathematical result about the architecture's own operators, ledger or data routing. Four of the six pass through Provenance (Layer 4): the ledger is the spine.

8.3 Provenance as the Spine

Four of the six bridges pass through Layer 4. The ledger's well-foundedness, proved once for evidence, is the same theorem that grounds a told story. The ledger read in order is a curriculum. The ledger read per slot is the market's funding. And Fit, the only source of lived mass, writes to the ledger by the Umwelt.

Provenance is not one layer among twelve. It is the structure the others are stated in. The history of what happened, and what it was worth, is the backbone from which ownership draws its funding, planning draws its curriculum, and selection draws its ground. Remove the provenance layer and four of the six bridges lose their footing.

The stand-alone modules (`Erlangen`, `Holonomy`, `Pacing`, `Market`) were initially disconnected from the SOV web. Bridge theorems were then written to state each result about the architecture's own operators and ledger. The unification is what those bridges revealed: provenance is the spine.

9 Related Work

Broad architectures of mind. Several architectures describe a mind across many faculties: ACT-R [Anderson et al., 2004], LIDA [Franklin et al., 2014], CLARION [Sun, 2016], NARS [Wang, 2013] and Sigma [Rosenbloom et al., 2016], with the Standard Model of the Mind as a consensus synthesis [Laird et al., 2017]; see Langley et al. [2009] and Kotseruba and Tsotsos [2020] for surveys. Their breadth is architectural: many components in one system. Sigma's "grand unification" is one shared computational substrate, not shared theorems. None of these is mechanized in a proof assistant, and none connects its faculties by proved bridges.

Single-principle theories unify by reduction: the free-energy principle accounts for perception, action and attention with one variational quantity [Friston, 2010], recast categorically by St Clere Smithe [2022]; the Conscious Turing Machine formalizes global-workspace attention as a competition for broadcast [Blum and Blum, 2022]; universal intelligence is fully formal but describes no faculties [Legg and Hutter, 2007].

Gerundelle unifies differently: not one substrate and not one principle, but named structures proved to hold across layers, connected by bridge theorems, and every result machine-checked.

Machine-checked agents. Agent verification is mature but targets programs, not minds. BDI agent programs are model-checked [Bordini et al., 2006, Dennis et al., 2012, Dennis, 2018], including the decision layer of autonomous systems [Dennis et al., 2016]; Luckcuck et al. [2019] survey the field. In a proof assistant, Jensen [2021, 2022] formalizes the GOAL agent language in Isabelle/HOL. McCann [2026] mechanizes in Coq the governance safety of AI workflow systems. Mechanizations of single components cover Markov decision processes [Vajjha et al., 2021, Schäffeler and Abdulaziz, 2023], AGM belief revision [Fouillard et al., 2021], and non-interference for hardware [Azevedo de Amorim et al., 2016].

None of these spans perception through narrative. Verifying an agent language proves that programs in that language behave correctly. Verifying an architecture of mind proves that the architecture's own layers compose correctly: that imagination cannot corrupt evidence, that perception determines what evidence is filed, that a told story is grounded in the history it tells. The levels of analysis are different.

Emotion and ownership. Logical formalizations of appraisal theory define emotions over belief, desire and ideals [Adam et al., 2009, Steunebrink et al., 2012]; EMA computes appraisal from plan structure [Gratch and Marsella, 2004]; and Reisenzein et al. [2013] argues for formal specification of emotion theories. Closest to reading affect off holdings, EBDI defines fear and anxiety over resources at risk [Pereira et al., 2008]. Active inference ties valence to the rate of change of the quantity perception minimizes [Joffily and Coricelli, 2013]. Mechanical checking of emotion models has so far been finite: answer-set verification of appraisal trajectories [Brännström and Nieves, 2025] and trace checking of a formalized model of Damasio [Bosse et al., 2008].

Gerundelle's contribution here is twofold. First, reading affect off the territory register rather than off beliefs and desires: the emotional phases are states of what the agent holds, not of what it thinks. Second, a proved map of which layer each phase reads: including the negative results (curiosity and anger do not factor through the register), which tell you exactly where the nice property breaks. We know of no prior proof-assistant mechanization of an affect model.

Narrative. Story generation by planning links events by causal structure [Riedl and Young, 2010, Ware and Young, 2014]; emergent narrative lets stories arise from simulated agents [Evans and Short, 2014]. Suspense has been modelled by escape plans [O'Neill and Riedl, 2014], by foreshadowing [Bae and Young, 2008], and as variance in an audience's beliefs [Ely et al., 2015]. Narrative has been verified before: Bosser et al. [2011] use Coq to prove properties of every plot variant of a story encoded in linear logic, and plot spaces have been model-checked [Thompson et al., 2015].

These verify a plot space a designer authored. Gerundelle proves well-foundedness of telling for every history of a simulated mind, and characterizes when a told story has value: suspense times foreshadowing, with a proved theorem that an unforeshadowed ending scores zero.

The prior work behind each individual result is credited in the *Prior work* paragraphs of sections 4 to 7.

	Symbolic AI	Statistical AI	Gerundelle
What is fixed	What is known: rules, axioms, ontologies	Architectural biases (attention, convolution); content is learned	The mechanics of knowing: operators, competition, provenance, territory
What is not known	A missing rule or a failed query: a gap with no shape	Diffuse probability over outcomes	A structural open slot: defined by its type and its relations to what is known, which narrow its feasible set; it pulls on attention
How it learns	Deduction, and knowledge written by hand	Global optimization over large corpora	Local funding: acting in a world and filing receipts
When the world contradicts it	Every conclusion resting on the changed premise is retracted and rebuilt (managed by truth maintenance [Doyle, 1979])	Further training; invariants and facts move together	The contradicted slot or box reopens (that revision stays confined to it is a design property, section 10)
Its own invariants (imagining is not seeing; hearsay is not experience)	Written as rules, if at all	Learned, and held statistically	Proved for every sequence (`firewall`, `took_told_le`)

Table 5: What each approach fixes in advance and what it leaves to experience.

10 Discussion

Why does unification matter? A guarantee proved about one layer in isolation says nothing about how the layers compose. The bridges are where composition is checked.

The firewall would be vacuous if the market's funding did not come from the ledger: what good is an imagination boundary if ownership draws on a different source? The story's well-foundedness would be a separate fact if sentences were not provenance entries: what grounds the story is the same structure that grounds evidence.

Stating the classical results on one architecture turns them from analogies into constraints on a single design. Non-interference in the literature is a property of security monitors. Here it is a property of imagination. Learning spaces in the literature describe what a student can learn next. Here they describe what a mind's own history teaches it. The mathematics is the same. The application is new, and the connections between applications are proved.

A system that explains things it was not designed to explain is a sign the abstraction is real. Each piece of Gerundelle was built for its own reasons: the story layer to tell what a mind did, the affect layer to read its territory, the ledger to keep its evidence honest. The bridges were written to connect them, and what they found was not built in: a told story's grounding is the ledger's own theorem, and affect has the same structure as perception. Pieces built for their own reasons chained into something none of them was designed for.

What the architecture fixes. Table 5 contrasts Gerundelle with symbolic and statistical AI by what each fixes in advance and what each leaves to experience.

The architecture in use. Fiction, code, embodiment and civilization run on the same architecture, because the same operations apply in each: push, observe the outcome, fund a slot, discover a condition, transfer by shape. The mind and the algebra stay fixed; only the world changes. The domains are projections, not separate systems. Five tracks run on the reference implementation; their results are empirical, measured in runs, not proved.

- **Civilization.** Populations of minds live in worlds whose physics is a parameter, and the civilization is whatever they produce there. A what-if question (what if there were no sunlight?) is a pair of runs that differ only in physics, and whatever institution, conflict or technology appears can be traced through provenance to the constraint that produced it. Fifteen rungs (P0 to P14) have been run, through populations, sub-wants, locks, grievance and retaliation.
- **Fiction.** A mind's encounter with a world, told by an author: fiction generated from first principles (a world, a mind, a ledger and author rules), with every sentence traceable to the world state that caused it, and no copyrighted text. The ladder's first eight rungs run from one character with one want to several endings held open; on every rung the reference configuration's told story reads back at 1.00, and the pipeline produces corpora.
- **Code.** The code is a mind, not something a mind writes: a program's structure is a graph of open variables, a deployed program is a mind with its slots closed, and a regression is the world waking it up. Ten rungs (0 to 9) have been run, through regression, transfer by shape and a population under scarcity.
- **Chess.** A player built on the same mind, in the world of a board. The first rungs run through constraint convergence on a back rank, in the player's own choice of move.
- **Embodiment.** Vision as a gerund field, space as bonded regions, and the body as the primary territory; this track is at the design stage.

Why formalize what already runs? Four reasons, in order of how surprising they were.

First, it finds bugs. The reference implementation passed its full test suite with the curiosity defect in place. Tests sample the sequences someone thought to try; a theorem quantifies over every sequence. The defect was in a function called every tick in every acting mind with phases enabled; it was wrong on every input its specification described; and no test caught it.

Second, it finds disagreements between design and code. Formalization recorded two places where the design documents and the implementation disagree: the surprise injected on contradiction (specified as full, implemented as 0.5 plus half the understanding), and warmth decay (specified as continuous, implemented as thresholded). It also measured the gap between the attestation discount's specification (schedule-independent) and its implementation (not: `reliability_order_dependent`).

Third, it sharpens the statement. The first guarantee proposed for affect was "two minds alike in ownership feel alike." It was false, as the code showed: curiosity reads the competition's winners, anger reads understanding. The true theorem was sharper and more useful: a proved map of which layer each phase reads.

Fourth, it makes revision inspectable. A mind with funded understanding adapts to a rule change by reopening only the slots that broke, while a neural network must retrain on everything, including what did not change. A contradiction here reopens a specific slot or stored box, not the whole system, and some of the conditions are proved: a told box reopens exactly when the world contradicts it (`told_reopens_iff`), and an eviction and a lived accrual on one slot do not commute (`accrue_evict_not_comm`). That revision stays confined to what broke is a property of the design; proving it in general is future work.

A note on learning. Learning in this architecture does not use backpropagation or gradient

descent. A slot is funded through interaction: receipts accumulate from lived trials, conditions are discovered by contrasting outcomes, a slot closes when evidence crosses the threshold, and it reopens when the world contradicts it. No gradient is computed and no weights are adjusted globally; each update is addressed to the slot the evidence concerns.

Limitations. These are honest, not hedged.

- **Every model is partial.** Section A lists what each layer's model leaves out.
- **Faithfulness is argued, not proved.** Section B lists every hypothesis that carries the claim that the model matches the code.
- **Exact vs. floating-point arithmetic.** The models compute over the reals; the implementation uses floating point. Conservation is proved exactly and asserted by the code to within 10^{-9}.
- **Flatness is proved on the complete graph only.** On a sparser graph of domains, every cycle (not only every triangle) must be flat.
- **The market bridge assumes one attestation discount**, while the ledger's attested receipts carry their own.
- **The round trip in selection is measured, not proved.** The determination system reads every sentence back at 1.00; this is empirical, not formal.
- **Operator minimality is syntactic.** The fiber signatures are pairwise distinct; whether one operator could be semantically simulated by others is a stronger question this does not settle.

Future work. The aim is every modality, beyond text and code. A modality is not built with modality-specific machinery: it falls out of which world the mind is in. Prose comes from a mind in a story world, programs from a mind in a code world, reaching from a mind with a body. Expanding to a new modality therefore means building the world it comes from, with the mind and the algebra unchanged. Each item connects to a track in the broader project, where the empirical work lives.

New worlds, new modalities.

- **Embodiment.** Vision as a gerund field, space as bonded regions, the body as the primary territory, and reaching. Its perception needs **distinctiveness space**: identity, orientation and reality-judgment by funded marks, with minimum-resolution profiles and the fake-detection property (the embodied track).
- **Images and video.** The world model is stepped forward in slices, and images and video come from the slices: the world model drives the modality, rather than the modality driving the model.
- **Civilization at population scale**: guarantees for populations of minds. The development already states grounding over one log shared by a population (`receiptsGrounded_applyAt`); the firewall, conservation in the population worlds, and the traceability of an institution to the constraint that produced it are next.
- **Every other modality** is, on this view, another world in which the mind can be placed.

What carries over, and what each world adds. Most of the guarantees in this paper are stated over the algebra and the mind, not over a domain: the firewall, provenance, the tournament, memory, the meta layer, ownership and affect do not mention text or code. They hold in any world whose operations fit the models. What each new world must supply is its own world layer (what it conserves, and how its doings change), its own perception (which transforms the agent cannot distinguish there), and its own read-back for the modality it produces, as the fiction pipeline reads each sentence back.

The formal program behind the existing worlds.

- **The fiction pipeline end to end**: the author's individual story rules, and the claim that every sentence of a generated corpus traces to the world state that caused it.
- **Code as a mind**: a regression stated as a contradiction that reopens only the slots that broke, and transfer by shape across programs.
- **Constraint convergence** mechanized: the `justOne` theorem generalized to the full grasp and chess settings, with simultaneous support in superposition.

Closing the remaining gaps in the models.

- **The world's phase changes** as threshold crossings: the mechanism the civilization ladder's conservation covers but the formalization does not yet.
- **The molecule's binding resolution** in competition, and the parked-box and affective-storage mechanisms in memory.
- **The full independence relation** of ledger events, of which accrual and eviction are one non-commuting pair.
- **Extraction or verified compilation**, so the implementation is the model and the gap in section B closes.

11 Conclusion

The standard way to represent knowledge is a chain of rules. The chain breaks when understanding changes. This paper exhibited an alternative (a twelve-layer architecture of mind built on a market of agents competing for scarce slots with empirical evidence) and proved that its guarantees hold.

The guarantees run from conservation of potential in the world to the well-foundedness of a told story. Bridge theorems connect them: stand-alone results from six fields stated about the architecture's own operators, ledger and data routing (perception and affect as one factorization property, and provenance the spine from which four other layers draw their ground).

Three things follow.

Every layer is verified. Not tested. Proved. For every input, every sequence, every interleaving.

The fields unify. Results from information security, the geometry of symmetry groups, fiber transport, learning theory, effect systems and provenance calculi (which rarely cite one another) are facets of one structure once instantiated on the architecture.

Perception and affect are one property. Both are factorization through a view: perception through the Umwelt, affect through the ownership register. Same mathematical structure. Two levels.

On unification. The mathematics underlying the layers was developed across six fields over more than a century, each for its own purpose: symmetry groups and their invariants [Klein, 1893], well-founded grounding [Kripke, 1975] and the support of truth maintenance [Doyle, 1979], non-interference [Goguen and Meseguer, 1982], knowledge spaces [Doignon and Falmagne, 1985], effect systems and graded monads [Lucassen and Gifford, 1988, Katsumata, 2014], and flat transport [Giraud, 1971]. These fields rarely cite one another. Whatever unification they admit was latent.

What formalization made explicit is that these fields, instantiated on this architecture's operators and ledger, describe the same system. The factorization property that governs what an agent can perceive (Layer 2) is the same universal property that determines which emotions the ownership register can produce (Layer 9). The well-foundedness proved for evidence chains (Layer 4) is the same well-foundedness that grounds every sentence of a told story (Layer 12). A valid curriculum (Layer 10) is the ledger read in prerequisite order: the history of evidence is the textbook, and every valid path through it is a shortest one.

None of this was designed into the code. Each piece of the reference implementation was built for its own reasons: the story layer to select narrative beats, the affect layer to read territory, the ledger to keep evidence honest. The shared structure became visible only when each layer's requirements were stated precisely, in Lean, and the bridge theorems were then written to prove it. That a told story's grounding is the ledger's well-foundedness theorem was found by formalizing the story layer; that affect shares perception's factorization was found by stating its readings exactly.

The contribution is not the invention of new mathematics. It is the recognition that six fields, complete on their own, share structure once stated about one cognitive architecture. A system that explains things it was not built to explain suggests the abstraction is real, not contingent on the builder's assumptions. When separately built solutions compose without having been designed to fit, the structure suggests itself as a property of the problem, not a choice. That these six fields converge here raises a question: how many other latent unifications are waiting, not for new mathematics, but for the formalization that makes them visible?

Code Availability. The Lean development (toolchain `v4.35.0`, Mathlib `v4.35.0`) and the reference implementation are available at https://github.com/ameritusweb/gerundelle. To check the development, run `lake build`. If it compiles, every theorem in this paper holds.

References

Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11):113036, 2011. doi: 10.1088/1367-2630/13/11/113036.

Carole Adam, Andreas Herzig, and Dominique Longin. A logical formalization of the OCC theory of emotions. *Synthese*, 168(2):201–248, 2009. doi: 10.1007/s11229-009-9460-9.

Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2):510–530, 1985. doi: 10.2307/2274239.

John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004. doi: 10.1037/0033-295X.111.4.1036.

Arthur Azevedo de Amorim, Nathan Collins, André DeHon, Delphine Demange, Cătălin Hriţcu, David Pichardie, Benjamin C. Pierce, Randy Pollack, and Andrew Tolmach. A verified information-flow architecture. *Journal of Computer Security*, 24(6):689–734, 2016. doi: 10.3233/JCS-15784.

Bernard J. Baars. *A Cognitive Theory of Consciousness*. Cambridge University Press, Cambridge, 1988. ISBN 0-521-30133-5.

Byung-Chull Bae and R. Michael Young. A use of flashback and foreshadowing for surprise arousal in narrative using a plan-based approach. In *Interactive Storytelling (ICIDS 2008)*, Lecture Notes in Computer Science, pages 156–167. Springer, 2008. doi: 10.1007/978-3-540-89454-4_22.

François Bancilhon and Nicolas Spyratos. Update semantics of relational views. *ACM Transactions on Database Systems*, 6(4):557–575, 1981. doi: 10.1145/319628.319634.

Eric B. Baum. Toward a model of intelligence as an economy of agents. *Machine Learning*, 35(2):155–185, 1999. doi: 10.1023/A:1007593124513.

Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Rémi Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, pages 1471–1479, 2016.

Lenore Blum and Manuel Blum. A theory of consciousness from a theoretical computer science perspective: Insights from the Conscious Turing Machine. *Proceedings of the National Academy of Sciences*, 119(21):e2115934119, 2022. doi: 10.1073/pnas.2115934119.

Rafael H. Bordini, Michael Fisher, Willem Visser, and Michael Wooldridge. Verifying multi-agent programs by model checking. *Autonomous Agents and Multi-Agent Systems*, 12(2):239–256, 2006. doi: 10.1007/s10458-006-5955-7.

Tibor Bosse, Catholijn M. Jonker, and Jan Treur. Formalisation of Damasio's theory of emotion, feeling and core consciousness. *Consciousness and Cognition*, 17(1):94–113, 2008. doi: 10.1016/j.concog.2007.06.006.

Anne-Gwenn Bosser, Pierre Courtieu, Julien Forest, and Marc Cavazza. Structural analysis of narratives with the Coq proof assistant. In *Interactive Theorem Proving (ITP 2011)*, volume 6898 of *Lecture Notes in Computer Science*, pages 55–70. Springer, 2011. doi: 10.1007/978-3-642-22863-6_7.

Robert Boyd and Peter J. Richerson. *Culture and the Evolutionary Process*. University of Chicago Press, Chicago, 1985.

Andreas Brännström and Juan Carlos Nieves. Human emotion verification by action languages via answer set programming. *Theory and Practice of Logic Programming*, 25(6):1047–1104, 2025. doi: 10.1017/S1471068426100416.

Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges, 2021.

Susan Carey. *Conceptual Change in Childhood*. MIT Press, Cambridge, MA, 1985. ISBN 0-262-03110-8.

Ernst Cassirer. The concept of group and the theory of perception. *Philosophy and Phenomenological Research*, 5(1):1–36, 1944. doi: 10.2307/2102891.

Taco Cohen and Max Welling. Group equivariant convolutional networks. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2990–2999. PMLR, 2016.

Eric Cosyn and Hasan B. Uzun. Note on two necessary and sufficient axioms for a well-graded knowledge space. *Journal of Mathematical Psychology*, 53(1):40–42, 2009. doi: 10.1016/j.jmp.2008.09.005.

Nelson Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001. doi: 10.1017/S0140525X01003922.

Antonio R. Damasio. *Descartes' Error: Emotion, Reason, and the Human Brain*. G. P. Putnam's Sons, New York, 1994.

Adnan Darwiche and Judea Pearl. On the logic of iterated belief revision. *Artificial Intelligence*, 89 (1–2):1–29, 1997. doi: 10.1016/S0004-3702(96)00038-0.

Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986. doi: 10.1016/0004-3702(86)90080-9.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Cham, 2021. Springer. doi: 10.1007/978-3-030-79876-5_37.

Edward L. Deci and Richard M. Ryan. *Intrinsic Motivation and Self-Determination in Human Behavior*. Plenum, New York, 1985. ISBN 0-306-42022-8.

Stanislas Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking, New York, 2014. ISBN 978-0-670-02543-5.

Louise A. Dennis. The MCAPL framework including the agent infrastructure layer and agent Java pathfinder. *Journal of Open Source Software*, 3(24):617, 2018. doi: 10.21105/joss.00617.

Louise A. Dennis, Michael Fisher, Matthew P. Webster, and Rafael H. Bordini. Model checking agent programming languages. *Automated Software Engineering*, 19(1):5–63, 2012. doi: 10.1007/s10515-011-0088-x.

Louise A. Dennis, Michael Fisher, Nicholas K. Lincoln, Alexei Lisitsa, and Sandor M. Veres. Practical verification of decision-making in agent-based autonomous systems. *Automated Software Engineering*, 23(3):305–359, 2016. doi: 10.1007/s10515-014-0168-9.

Jean-Paul Doignon and Jean-Claude Falmagne. Spaces for the assessment of knowledge. *International Journal of Man-Machine Studies*, 23(2):175–196, 1985. doi: 10.1016/S0020-7373(85)80031-6.

Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979. doi: 10.1016/0004-3702(79)90008-0.

Jeffrey Ely, Alexander Frankel, and Emir Kamenica. Suspense and surprise. *Journal of Political Economy*, 123(1):215–260, 2015. doi: 10.1086/677350.

Richard Evans and Emily Short. Versu—a simulationist storytelling system. *IEEE Transactions on Computational Intelligence and AI in Games*, 6(2):113–130, 2014. doi: 10.1109/TCIAIG.2013.2287297.

Jean-Claude Falmagne and Jean-Paul Doignon. *Learning Spaces: Interdisciplinary Applied Mathematics*. Springer, Berlin, Heidelberg, 2011. ISBN 978-3-642-01038-5. doi: 10.1007/978-3-642-01039-2.

J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007. doi: 10.1145/1232420.1232424.

Valentin Fouillard, Safouan Taha, Frédéric Boulanger, and Nicolas Sabouret. Belief revision theory. *Archive of Formal Proofs*, October 2021. ISSN 2150-914x. https://isa-afp.org/entries/Belief_Revision.html, Formal proof development.

Stan Franklin, Tamas Madl, Sidney D'Mello, and Javier Snaider. LIDA: A systems-level architecture for cognition, emotion, and learning. *IEEE Transactions on Autonomous Mental Development*, 6 (1):19–41, 2014. doi: 10.1109/TAMD.2013.2277589.

Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11 (2):127–138, 2010. doi: 10.1038/nrn2787.

Soichiro Fujii, Shin-ya Katsumata, and Paul-André Melliès. Towards a formal theory of graded monads. In *Foundations of Software Science and Computation Structures (FoSSaCS 2016)*, volume 9634 of *Lecture Notes in Computer Science*, pages 513–530. Springer, 2016. doi: 10.1007/978-3-662-49630-5_30.

Tingran Gao, Jacek Brodzki, and Sayan Mukherjee. The geometry of synchronization problems and learning group actions. *Discrete & Computational Geometry*, 65(1):150–211, 2021. doi: 10.1007/s00454-019-00100-2.

Allen Van Gelder, Kenneth A. Ross, and John S. Schlipf. The well-founded semantics for general logic programs. *Journal of the ACM*, 38(3):619–649, 1991. doi: 10.1145/116825.116838.

James J. Gibson. *The Ecological Approach to Visual Perception*. Psychology Press, 2014. doi: 10.4324/9781315740218. Classic edition; originally published 1979.

Jean Giraud. *Cohomologie non abélienne*, volume 179 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1971. doi: 10.1007/978-3-662-62103-5.

Joseph A. Goguen and José Meseguer. Security policies and security models. In *Proceedings of the 1982 IEEE Symposium on Security and Privacy*, pages 11–20, Oakland, CA, USA, 1982. IEEE Computer Society. doi: 10.1109/SP.1982.10014.

Jonathan Gratch and Stacy Marsella. A domain-independent framework for modeling emotion. *Cognitive Systems Research*, 5(4):269–306, 2004. doi: 10.1016/j.cogsys.2004.02.002.

Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS '07)*, pages 31–40, Beijing, China, 2007. ACM. doi: 10.1145/1265530.1265535.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, New York, NY, USA, 2023. ACM. doi: 10.1145/3605764.3623985.

Jakob Hansen and Robert Ghrist. Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology*, 3(4):315–358, 2019. doi: 10.1007/s41468-019-00038-7.

John M. Howie. The subsemigroup generated by the idempotents of a full transformation semigroup. *Journal of the London Mathematical Society*, s1-41(1):707–716, 1966. doi: 10.1112/jlms/s1-41.1.707.

Qi-Xing Huang and Leonidas Guibas. Consistent shape maps via semidefinite programming. *Computer Graphics Forum*, 32(5):177–186, 2013. doi: 10.1111/cgf.12184.

Alexander Birch Jensen. Towards verifying GOAL agents in Isabelle/HOL. In *Proceedings of the 13th International Conference on Agents and Artificial Intelligence (ICAART)*, pages 345–352. SciTePress, 2021. doi: 10.5220/0010268503450352.

Alexander Birch Jensen. Machine-checked verification of cognitive agents. In *Proceedings of the 14th International Conference on Agents and Artificial Intelligence (ICAART), Volume 1*, pages 245–256. SciTePress, 2022. doi: 10.5220/0010838700003116.

Mateus Joffily and Giorgio Coricelli. Emotional valence and the free-energy principle. *PLoS Computational Biology*, 9(6):e1003094, 2013. doi: 10.1371/journal.pcbi.1003094.

Audun Jøsang. *Subjective Logic: A Formalism for Reasoning Under Uncertainty*. Artificial Intelligence: Foundations, Theory, and Algorithms. Springer, 2016. doi: 10.1007/978-3-319-42337-1.

Annette Karmiloff-Smith. *Beyond Modularity: A Developmental Perspective on Cognitive Science*. MIT Press, Cambridge, MA, 1992. ISBN 0-262-11169-1.

Shin-ya Katsumata. Parametric effect monads and semantics of effect systems. In *Proceedings of the 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '14)*, pages 633–645, San Diego, CA, USA, 2014. ACM. doi: 10.1145/2535838.2535846.

Felix Klein. A comparative review of recent researches in geometry. *Bulletin of the New York Mathematical Society*, 2(10):215–249, 1893. doi: 10.1090/S0002-9904-1893-00147-X. Translated by M. W. Haskell.

Iuliia Kotseruba and John K. Tsotsos. 40 years of cognitive architectures: core cognitive abilities and practical applications. *Artificial Intelligence Review*, 53(1):17–94, 2020. doi: 10.1007/s10462-018-9646-y.

Saul Kripke. Outline of a theory of truth. *The Journal of Philosophy*, 72(19):690–716, 1975. doi: 10.2307/2024634.

Maria E. Kronfeldner. Is cultural evolution Lamarckian? *Biology & Philosophy*, 22(4):493–512, 2007. doi: 10.1007/s10539-006-9037-7.

Thomas S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, Chicago, 1962.

John E. Laird, Christian Lebiere, and Paul S. Rosenbloom. A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *AI Magazine*, 38(4):13–26, 2017. doi: 10.1609/aimag.v38i4.2744.

Pat Langley, John E. Laird, and Seth Rogers. Cognitive architectures: Research issues and challenges. *Cognitive Systems Research*, 10(2):141–160, 2009. doi: 10.1016/j.cogsys.2006.07.004.

Richard S. Lazarus. *Emotion and Adaptation*. Oxford University Press, New York, 1991. ISBN 0-19-506994-3.

Shane Legg and Marcus Hutter. Universal intelligence: A definition of machine intelligence. *Minds and Machines*, 17(4):391–444, 2007. doi: 10.1007/s11023-007-9079-x.

E. L. Lehmann and Joseph P. Romano. *Testing Statistical Hypotheses*. Springer Texts in Statistics. Springer, New York, 3rd edition, 2005. doi: 10.1007/0-387-27605-X.

George Loewenstein. The psychology of curiosity: A review and reinterpretation. *Psychological Bulletin*, 116(1):75–98, 1994. doi: 10.1037/0033-2909.116.1.75.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '88)*, pages 47–57. ACM, 1988. doi: 10.1145/73560.73564.

Matt Luckcuck, Marie Farrell, Louise A. Dennis, Clare Dixon, and Michael Fisher. Formal specification and verification of autonomous robotic systems: A survey. *ACM Computing Surveys*, 52(5):1–41, 2019. doi: 10.1145/3342355.

McCann. Mechanized foundations of structural governance: Machine-checked proofs for governed intelligence, 2026.

L. David Mech. Alpha status, dominance, and division of labor in wolf packs. *Canadian Journal of Zoology*, 77(8):1196–1203, 1999. doi: 10.1139/z99-099.

George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956. doi: 10.1037/h0043158.

Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4): 541–580, 1989. doi: 10.1109/5.24143.

Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. seL4: From general purpose to a proof of information flow enforcement. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy*, pages 415–429. IEEE, 2013. doi: 10.1109/SP.2013.35.

Susan Nolen-Hoeksema. Responses to depression and their effects on the duration of depressive episodes. *Journal of Abnormal Psychology*, 100(4):569–582, 1991. doi: 10.1037/0021-843X.100.4.569.

Brian O'Neill and Mark Riedl. Dramatis: A computational model of suspense. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 28(1), 2014. doi: 10.1609/aaai.v28i1.8836.

Dominic Orchard and Tomas Petricek. Embedding effect systems in Haskell. In *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, pages 13–24. ACM, 2014. doi: 10.1145/2633357.2633368.

Pierre-Yves Oudeyer, Frédéric Kaplan, and Verena V. Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE Transactions on Evolutionary Computation*, 11(2): 265–286, 2007. doi: 10.1109/TEVC.2006.890271.

Deepti Pachauri, Risi Kondor, and Vikas Singh. Solving the multi-way matching problem by permutation synchronization. In *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, pages 1860–1868, 2013.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2778–2787. PMLR, 2017.

David Pereira, Eugénio Oliveira, and Nelma Moreira. Formal modelling of emotions in BDI agents. In *Computational Logic in Multi-Agent Systems (CLIMA VIII)*, Lecture Notes in Computer Science, pages 62–81. Springer, 2008. doi: 10.1007/978-3-540-88833-8_4.

Jean Piaget. *The Origins of Intelligence in Children*. International Universities Press, New York, 1952.

Rainer Reisenzein, Eva Hudlicka, Mehdi Dastani, Jonathan Gratch, Koen Hindriks, Emiliano Lorini, and John-Jules Ch. Meyer. Computational modeling of emotion: Toward improving the inter- and intradisciplinary exchange. *IEEE Transactions on Affective Computing*, 4(3):246–266, 2013. doi: 10.1109/T-AFFC.2013.14.

Mark O. Riedl and R. Michael Young. Narrative planning: Balancing plot and character. *Journal of Artificial Intelligence Research*, 39:217–268, 2010. doi: 10.1613/jair.2989.

Paul S. Rosenbloom, Abram Demski, and Volkan Ustun. The Sigma cognitive architecture and system: Towards functionally elegant grand unification. *Journal of Artificial General Intelligence*, 7(1):1–103, 2016. doi: 10.1515/jagi-2016-0001.

Richard M. Ryan and Edward L. Deci. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1):68–78, 2000. doi: 10.1037/0003-066X.55.1.68.

Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi: 10.1109/JSAC.2002.806121.

Maximilian Schäffeler and Mohammad Abdulaziz. Formally verified solution methods for Markov decision processes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37(12), pages 15073–15081, 2023. doi: 10.1609/aaai.v37i12.26759.

Klaus R. Scherer. Appraisal considered as a process of multilevel sequential checking. In Klaus R. Scherer, Angela Schorr, and Tom Johnstone, editors, *Appraisal Processes in Emotion: Theory, Methods, Research*, pages 92–120. Oxford University Press, New York, 2001. doi: 10.1093/oso/9780195130072.003.0005.

Jürgen Schmidhuber. Curious model-building control systems. In *Proceedings of the 1991 IEEE International Joint Conference on Neural Networks*, volume 2, pages 1458–1463, Singapore, 1991. doi: 10.1109/IJCNN.1991.170605.

Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems 30*, 2017.

Amit Singer. Angular synchronization by eigenvectors and semidefinite programming. *Applied and Computational Harmonic Analysis*, 30(1):20–36, 2011. doi: 10.1016/j.acha.2010.02.001.

Toby St Clere Smithe. *Mathematical Foundations for a Compositional Account of the Bayesian Brain*. PhD thesis, University of Oxford, 2022.

Bas R. Steunebrink, Mehdi Dastani, and John-Jules Ch. Meyer. A formal model of emotion triggers: an approach for BDI agents. *Synthese*, 185(Suppl. 1):83–129, 2012. doi: 10.1007/s11229-011-0004-8.

Alexander L. Strehl and Michael L. Littman. An analysis of model-based interval estimation for Markov decision processes. *Journal of Computer and System Sciences*, 74(8):1309–1331, 2008. doi: 10.1016/j.jcss.2007.08.009.

Ron Sun. *Anatomy of the Mind: Exploring Psychological Mechanisms and Processes with the Clarion Cognitive Architecture*. Oxford University Press, 2016. doi: 10.1093/acprof:oso/9780199794553.001.0001.

Leonard Talmy. Force dynamics in language and cognition. *Cognitive Science*, 12(1):49–100, 1988. doi: 10.1207/s15516709cog1201_2.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, New Orleans, LA, USA, 2020. ACM. doi: 10.1145/3372885.3373824.

Matt Thompson, Steve Battle, and Julian Padget. Telling non-linear stories with interval temporal logic. In *Interactive Storytelling (ICIDS 2015)*, Lecture Notes in Computer Science, pages 370–373. Springer, 2015. doi: 10.1007/978-3-319-27036-4_39.

Koundinya Vajjha, Avraham Shinnar, Barry Trager, Vasily Pestun, and Nathan Fulton. CertRL: formalizing convergence proofs for value and policy iteration in Coq. In *Proceedings of the 10th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2021)*, pages 18–31, 2021. doi: 10.1145/3437992.3439927.

Dennis Volpano, Cynthia Irvine, and Geoffrey Smith. A sound type system for secure flow analysis. *Journal of Computer Security*, 4(2–3):167–187, 1996. doi: 10.3233/JCS-1996-42-304.

Jakob von Uexküll. *A Foray into the Worlds of Animals and Humans, with A Theory of Meaning*. Posthumanities. University of Minnesota Press, Minneapolis, 2010. ISBN 978-0-8166-5900-5. Translated by Joseph D. O'Neil; introduction by Dorion Sagan; afterword by Geoffrey Winthrop-Young.

L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1978. ISBN 0-674-57629-2. Edited by Michael Cole, Vera John-Steiner, Sylvia Scribner and Ellen Souberman.

Matthew P. Walker and Robert Stickgold. Sleep-dependent learning and memory consolidation. *Neuron*, 44(1):121–133, 2004. doi: 10.1016/j.neuron.2004.08.031.

Pei Wang. *Non-Axiomatic Logic: A Model of Intelligent Reasoning*. World Scientific, 2013. doi: 10.1142/8665.

Stephen G. Ware and R. Michael Young. Glaive: A state-space narrative planner supporting intentionality and conflict. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 10(1), pages 80–86, 2014. doi: 10.1609/aiide.v10i1.12712.

Michael P. Wellman. A market-oriented programming environment and its application to distributed multicommodity flow problems. *Journal of Artificial Intelligence Research*, 1:1–23, 1993. doi: 10.1613/jair.2.

Alfred North Whitehead. *Process and Reality: An Essay in Cosmology*. Macmillan, New York, 1929.

Kenneth G. Wilson. Confinement of quarks. *Physical Review D*, 10(8):2445–2459, 1974. doi: 10.1103/PhysRevD.10.2445.

Bluma Zeigarnik. Das Behalten erledigter und unerledigter Handlungen. *Psychologische Forschung*, 9 (1):1–85, 1927. doi: 10.1007/BF02409755. Untersuchungen zur Handlungs- und Affektpsychologie III, edited by Kurt Lewin.

Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. Can LLMs separate instructions from data? and what do we even mean by that? In *International Conference on Learning Representations (ICLR)*, 2025.

A Coverage

The models are not the code. Each layer's Lean development captures the code's invariants but leaves parts of the implementation unmodelled (cases that are too implementation-specific to state as general guarantees, or that lie outside the current development's scope). The table below makes these gaps explicit. A gap is not a weakness in the proved guarantees; it is an invitation to extend them.

Layer	Not covered by the models
1 World	ForceWorld, which does not conserve; phase changes as threshold crossings. Some moves are written as assignments in the code; they are transfers whenever the code reaches them.
2 Perception	The distinctiveness space.
3 Competition	The resolution of each molecule's binding; the order is any sorting by a key.
4 Provenance	The market uses one discount c_0, while the ledger's attested receipts carry their own; the market bridge matches exactly when all are equal.
5 Memory	Parked boxes, rationalization, the storage of affective phases.
6 Understanding	Flatness is the complete-graph case. Operator minimality is a syntactic separation of fiber signatures.
7 Self-awareness	Predictions decided by outcomes (conflated, distinction, same) are outside the deadline theorem.
8 Ownership	Holder states, hysteresis and half-lives; ties are broken by any rule.
9 Affect	The two anger sources outside the register are taken as the sets of agents they blame.
10 Planning	The order in which a sub-want is chosen; the option to restore a dropped fact.
11 Social	Trust is modelled as the code's moving average; the specification's schedule-independent decay is not.
12 Selection	The individual story rules; the reader's round trip, which is measured; entropies are taken as given.

B Modelling Hypotheses

A Lean theorem holds of its definitions. Whether those definitions faithfully model the code is a separate question (one the proof assistant cannot answer, because it cannot read Python). We answer it the only way available: by inspection, by test, and by argument, listed here for every hypothesis that carries weight.

Two kinds of hypothesis appear in the development. The first kind is a condition on the theorem's inputs (curiosity at most 1, $c_0 \in [0, 1]$) stated in the theorem block and inherited by the reader. The second kind stands in for a property of the reference implementation, established outside the proof.

The table below lists only the second kind. For each one: what the hypothesis assumes, and how that assumption was established.

Theorem	Hypothesis standing in for the code	How it was established
`nonneg_runMoves`	Every move takes a nonnegative amount, at most what its source holds.	Read: `spend` refuses when the store is short; a draw takes the minimum of draw, capacity and field; spill and landing move exactly what is held.
`total_runMoves`	The world's moves are transfers between places.	Argued: the code writes forming, propelling and the parts' shares as assignments, which equal transfers when the target place is empty, as it is wherever the code reaches them.
`winner_outbids`, `operable_win_first`	The tournament's order permutes its input and sorts it by a key.	Read: `think` sorts by bid, surprise, time waited and id, a total order.
`bid_ignores_meta`	A thought's bid is a function of its admissible partners and of the constraint molecules supporting it.	Read: `resolve`'s candidate filter, `support`, `tension` and `forward`, which read only the molecule, its own binding and the understanding and territory registers. A bond chooses its edge among the most active carriers of its end tags; the code now excludes meta molecules from that choice, a guard added for this correspondence that changed no test result in either copy of the implementation.
`check_before_deadline`	`holds` returns a verdict once the deadline is reached.	Read each branch of `holds`: true for the seven kinds checked by tick, not for conflated, distinction and same, which the theorem therefore does not cover.
`inv_run`, `drops_le_card`	A sub-want is set only below depth four and only if it is not the current want, on the stack, or dropped before.	Read: `planned`'s depth guard and `subwant`'s `makeable`. The option `keep_subs`, off by default, is not modelled.
`sourced_rebuild`	Each pass keeps a reference only if it is before a bound at or before the plan's position.	Argued: the code compares with `out.index(p)`, the first plan equal to p, which is at or before p's position.

Theorem	Hypothesis standing in for the code	How it was established
`took_trials`	One attestation discount c_0 for every told trial.	Not true in general: the ledger's attested receipts each carry their own discount. The bridge is exact when they are equal.
Affect theorems	`Phases.graph` as translated; anger's sources outside the register as the sets of agents they blame.	Translated line by line, after the curiosity correction (section C); the anger sources are a modelling choice.
Memory theorems	Reopening by resonance, by a hole, or by contradiction; told boxes have no keys.	Read: `reopen`, `contradict` and `tell`. Parked boxes are not modelled.
`held_nonneg`, `value_pos_iff`	The reader's entropies are nonnegative.	A property of entropy, not of the code.

Table 7: Hypotheses that stand in for properties of the reference implementation.

Exact and floating-point arithmetic. The models compute over the reals; the implementation uses floating point. Conservation of potential is proved exactly, while the code asserts it to within 10^{-9} after each slice. Warmth, bids and pulls are likewise exact in the models and rounded in the code. The guarantees are about the models' arithmetic (which is what you want from a guarantee): the real number is the standard the floating-point implementation is held to.

Results that hold by construction. Some guarantees follow closely from how the model is built: the firewall holds because register moves cannot touch the ledger, and a told box reopens only by contradiction because it has no keys. They are stated as guarantees anyway, because the design depends on them. A property the design depends on is worth proving, even when the proof is short.

C The Curiosity Correction

The introduction described the curiosity defect in outline (section 1). This appendix shows the mechanics: what the code did, what the correction changed, and what moved in the test results.

The function `Phases.graph` reads emotional phases off the ownership register each tick. Its curiosity branch checks whether an unclaimed slot is among the competition's winners. Before the correction, that check sat in a loop over *parked* slots (which can never win anything) and read a stale variable `inst` left over from the previous live-slot loop. The check could be true for the wrong slot (the stale `inst`), never for the parked slot it was supposed to test.

Algorithm 1 The curiosity reading in `Phases.graph`, before and after

for each live slot (k, r) **do** {$inst \leftarrow$ own:k:r}
  read frustration, anxiety, sadness on *inst*
  if $k =$ unclaimed **and** *inst* $\in$ top **then** form curiosity on *inst* {after: here}
for each parked slot (k, r) **do** {never among the winners}
  read sadness on a lost slot opening while parked
  if $k =$ unclaimed **and** *inst* $\in$ top **then** form curiosity on *inst* {before: here, with *inst* stale}

Moving the check to line 3, where `inst` is fresh, is the entire correction. The corrected Python and its Lean model, side by side:

```
for k, r in sorted(live):
    row, inst = terr.rows.get(r, {}), \
                f"own:{k}:{r}"
    ...
    if k == "unclaimed" and inst in top:
        refreshed.add(
            self.form("curiosity", inst, rec))
```

```
def curiosityReading (reg : Register R A)
    (ctx : Context R A) : Set (Phase R A) :=
  {p | ∃ r,
     (SlotKind.unclaimed, r) ∈ reg.live ∧
     (SlotKind.unclaimed, r) ∈ ctx.top ∧
     p = (.curiosity, .own .unclaimed r)}
```

The loop over live slots becomes the condition $(\mathit{unclaimed}, r) \in \mathit{reg.live}$; the winners `top` become the context's `ctx.top`; and the formed phase becomes a member of the returned set. Before the correction, no Lean definition of this shape could match the code. That mismatch is how the defect was found: translation demanded a definition, and no correct definition existed.

The test suite consists of behavioural scenarios in simulated force worlds; several run over five random seeds, reported as the number of seeds that pass. The suite passed 72 of 80 tests (2 pending) before and after the correction, and no test changed status. The seeded scenarios whose results moved all improved:

- T60, *spring and raised weight* (pending): after one mind lives through a spring world and then a weight world, its understanding forms one abstract structure in which the push on the clamp and the push on the catch play the same role; 0 to 2 of 5 seeds.
- T62, *paired lights*: a hidden driver lights lamp A, then lamp B. With A's plug unseated by the agent, B still lights, and the agent holds no link from A to B as caused, only as observed; 4 to 5 of 5.
- T66, *three enabling systems* (pending): after a powered button, a safety catch and a guarded file, one abstract structure holding a condition on an action has pushes from at least two of the domains in that action's role; 4 to 5 of 5.
- T82, *same action, different goal*: a neighbour's goal depends on the scene, and the agent reads two purposes behind the same kind of move, split by the scene; 4 to 5 of 5.
- T68, *goals*: the number of pushes the planner needed to reach the goal fell on two tasks, a lock from 6 to 2 and an assembly from 7 to 4. Fewer pushes means the goal was reached more efficiently.

The same changes appeared in both copies of the implementation. A function wrong on every input its specification described failed no test. A proof assistant found what the suite could not.

D The Formal Development

File	Thm.	File	Thm.	File	Thm.
`Affect`	13	`Firewall`	5	`Monad`	25
`Bridges`	12	`Geometry`	17	`Operator`	22
`Competition`	8	`Holonomy`	10	`Pacing`	7
`Confluence`	15	`Integrity`	5	`Planning`	7
`Crypto`	3	`Ledger`	22	`Provenance`	20
`Discount`	12	`Market`	21	`Quotient`	22
`Erlangen`	12	`Memory`	6	`Selection`	9
		`Meta`	16	`Slot`	5
				`World`	7

Total: 24 files, 301 theorems, about 4,350 lines; no `sorry`, `admit` or `axiom` declarations; only the axioms `propext`, `Classical.choice` and `Quot.sound`.

Table 8: Theorems per Lean file.